



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Programmation Orientée Objet en Java

*Progression pour découvrir les concepts objet
et les implémenter en Java*

Sommaire

Préambule.....	4
1. Présentation de la Programmation Orienté Objet POO	5
1.1. Programmation procédurale Vs orientée Objet	5
1.1.1. Programmation procédurale :	5
1.1.2. Programmation Orientée Objet	5
2. Présentation de UML	6
2.1. Uml qu'est-ce que c'est ?	6
2.2. Historique :	6
3. Présentation de Java	8
4. Les Objets	9
4.1. Présentation de l'Objet	9
4.2. Un Exemple	9
4.2.1. Quels sont les objets de ce domaine ?	9
4.2.2. Les objets du domaine.....	10
4.2.3. Le comportement de nos objets Personne	11
4.2.4. Nos objets Personne utilisés par une application	12
4.2.5. Les diagrammes d'interactions de UML.....	13
5. Classe d'objet	14
5.1. Objectifs	14
5.1.1. Notion de classe, de propriété, de méthode	14
5.1.2. Les Classes en UML	15
5.1.3. Définition d'une classe en Java	17
5.1.4. Instanciation d'une classe	20
6. Encapsulation – Protection et accès aux données	27
6.1. Objectifs	27
6.2. Ce qu'il faut savoir.....	27
6.2.1. Protection des données.....	27
6.2.2. Accesseurs et Modifieurs	27
6.2.3. Abstraction Procédurale	28
6.2.4. Encapsulation	29
6.2.5. Types énumérés.....	30
7. Les liens entre objets.....	34
7.1. Le contexte	34
7.2. Représentation dans le diagramme d'objet UML.....	34
7.3. Représentation dans le diagramme de classes UML	34
7.3.1. Associations	34
7.3.1. Multiplicités	35
Rôles.....	35
7.3.2. Associations Unidirectionnelles vs Bidirectionnelles.....	36
7.4. Implémentation des associations	37
7.4.1. Associations One To Many ou Many To One	37
7.4.2. Associations Many To Many	38
7.4.3. Associations Many To Many avec une classe d'association	39
Conclusion :	39
8. Notions diverses de POO	41
8.1. Objectifs	41

8.2.	Propriétés et méthodes d'instance vs de classe	41
8.2.1.	Propriété de classe.....	41
8.2.2.	Méthode de classe	42
8.3.	Les objets sont des données comme les autres	43
8.4.	Les destructeurs	43
9.	Héritage.....	45
9.1.	Objectifs	45
9.2.	Ce qu'il faut savoir.....	45
9.2.1.	L'héritage.....	45
9.2.2.	Représentation de l'héritage en UML :	46
9.2.3.	Implémentation de l'héritage en Java :	46
9.2.4.	Exemple basé sur « Personne » :.....	47
9.2.5.	Visibilité des propriétés et des méthodes dans la classe Fille.....	49
9.2.6.	Constructeurs dans une classe dérivée (sous classe).....	49
9.2.7.	Redéfinition des méthodes de la classe Parente.....	49
9.2.8.	Type statique vs Type dynamique :	51
9.2.9.	Compatibilité et affectation, casting	52
9.2.10.	Classe de base.....	53
9.2.11.	Conclusion :	53
	Le Polymorphisme	54
9.3.	Objectifs	54
9.4.	Ce qu'il faut savoir.....	54
10.	Classe Abstraite et interfaces	56
10.1.	Objectifs.....	56
10.2.	Ce qu'il faut savoir	56
10.2.1.	Classe abstraite	56
10.2.2.	Classes abstraite en UML	56
10.2.3.	Classes abstraite en Java.....	57
10.2.4.	Méthode abstraite	58
10.2.5.	Interface.....	59
11.	Collectionner des objets	61
11.1.	Objectifs.....	61
11.2.	Ce qu'il faut savoir	61
11.2.1.	Les tableaux.....	61
11.2.2.	Les collections.....	62
12.	Exceptions.....	65
12.1.	Objectifs.....	65
12.2.	Ce qu'il faut savoir	65
12.2.1.	La classe Exception et ses filles	65
12.2.2.	Lever une exception	65
12.2.3.	Intercepter et traiter une exception.....	66
12.2.4.	Créer une nouvelle classe Exception.....	67
	Index.....	68

Préambule

Ce support présente la POO et un peu UML, il comporte des parties de cours, des liens vers des vidéos pédagogiques et un ensemble de TP permettant de découvrir les concepts de la Programmation Orientée Objet et de les mettre en œuvre.

Des bases en algorithmique et une connaissance de la programmation procédurale sont un prérequis.

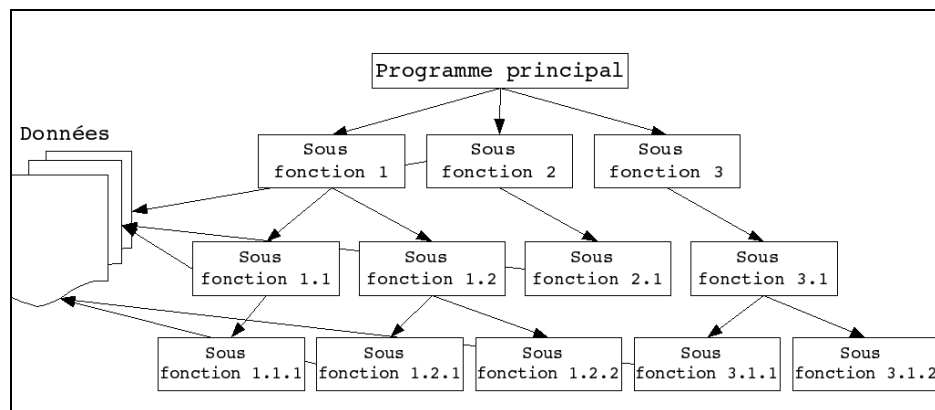
Les trois points principaux du paradigme objets : l'encapsulation, l'héritage et le polymorphisme seront abordés.

1. Présentation de la Programmation Orienté Objet POO

1.1. Programmation procédurale Vs orientée Objet

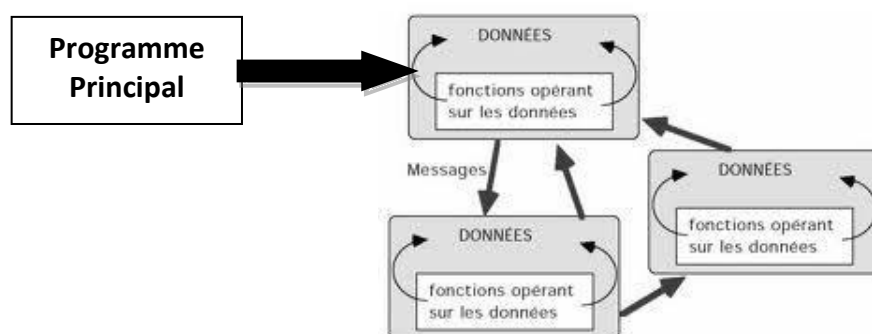
1.1.1. Programmation procédurale :

- Le programme est divisé en procédures et fonctions qui s'utilisent les unes les autres.
- La création du programme est orienté « traitements ». La tâche à réaliser a été découpée en sous-tâches
- Les données manipulées sont globales et partagées entre toutes les fonctions.
- Le programme principal (Main) utilise les fonctions pour réaliser la tâche attendue.



1.1.2. Programmation Orientée Objet

- Le programme est divisé en objets qui s'utilisent les uns les autres en communiquant par « Messages ».
- Le découpage en objets est déduit de la structure du problème à résoudre (l'environnement, le domaine de de la tâche à réaliser). Ce découpage n'est pas dirigé par les données ou les traitements mais par les objets qui représentent les entités du domaine.
- Chaque objet comporte une partie des données du domaine, celles qui lui sont propres, et des opérations (fonctions, méthodes) qui constituent son comportement.



2. Présentation de UML

2.1. Uml qu'est-ce que c'est ?

UML permet de modéliser et de représenter graphiquement les systèmes dans le domaine du développement logiciel et particulièrement en orienté objet.

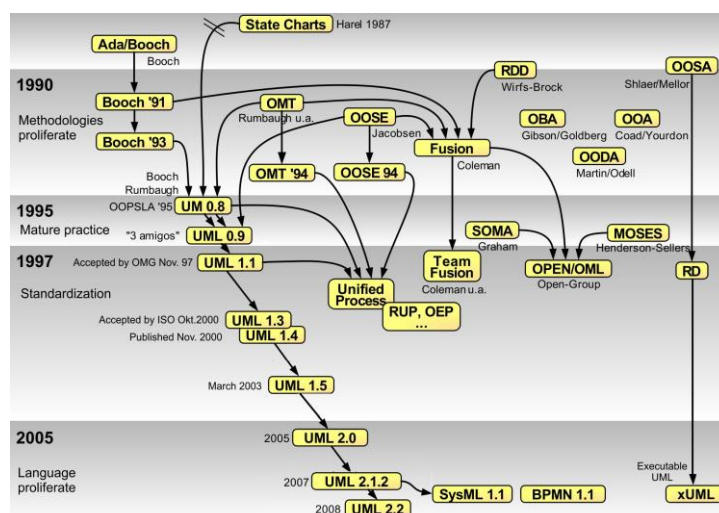
Le langage UML (Unified Modeling Language, ou langage de modélisation unifié) a été pensé pour être un langage de modélisation visuelle commun, et riche sémantiquement et syntaxiquement. Il est destiné à l'architecture, la conception et la mise en œuvre de systèmes logiciels complexes par leur structure aussi bien que leur comportement. L'UML a des applications qui vont au-delà du développement logiciel. Il est utilisé pour concevoir et représenter des systèmes dans de nombreux domaines :

- Système d'information des entreprises
- Modélisation des processus (Services, Organisations, WorkFlow etc.)
- Télécommunication (structure de réseaux, protocoles)
- Sciences (modélisation)
- UML n'est pas une méthode, c'est un langage
- UML est normalisé par l'OMG
- UML est dans le domaine public (utilisable sans licence)
- UML peut être compris par des machines (MDA)

2.2. Historique :

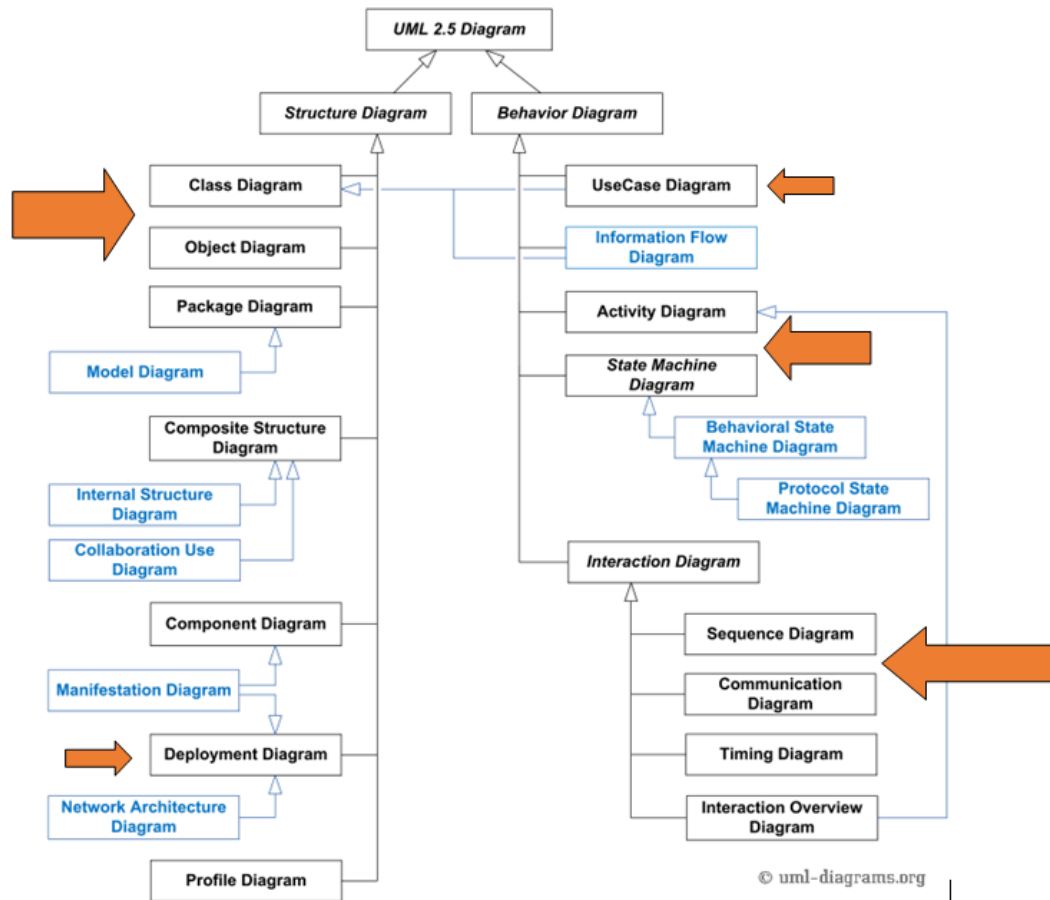
Depuis le milieu des années 80 les méthodes et langages de modélisation objet se sont multipliées. En 1995 il existait plus de 50 méthodes et langages différents : plus personne ne peut communiquer. [https://fr.wikipedia.org/wiki/UML_\(informatique\)#Histoire](https://fr.wikipedia.org/wiki/UML_(informatique)#Histoire)

En 1995, **James Rumbaugh** auteur de la méthode OMT avec **Grady Booch** auteur de la méthode BOOCH et **Ivar Jacobson** créateur des cas d'utilisation coopèrent au sein de la société Rational Rose et créent la première version d'UML.

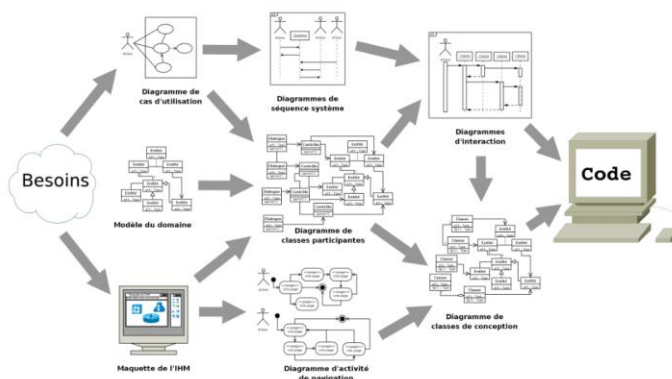


By Guido Zockoll, Axel Scheithauer Marcel Douwe Dekker (Mdd) - Translation and update of File:OO-historie-2.svg by AxelScheithauer, Okt 6, 2009, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=23052855>

La dernière version d'UML est la 2.5 de décembre 2017.



Nous apprendrons à lire ces diagrammes plutôt qu'à les concevoir.



Il est possible de déduire du code à partir de certains diagrammes UML.

Visual Paradigm : un modelleur UML simple et ergonomique :

<https://www.visual-paradigm.com/download/community.jsp>

3. Présentation de Java

Présentation

- ▶ **Java** est un langage de programmation orienté objet
- ▶ Créé par James Gosling et Patrick Naughton, employés de **Sun Microsystems** (présentation officielle en 1995)
- ▶ Depuis 2009 **Oracle** a racheté **Sun** et est propriétaire de **Java**

Objectifs du langage (2)

- ▶ Très performant
 - ▶ Pas le cas au début car interprétation du bytecode (lenteur)
 - ▶ Compilation JIT (à la volée)
- ▶ Multitâches et dynamique
 - ▶ Support du Multi-thread
 - ▶ Réflexion

Objectifs du langage (1)

- ▶ Simple, orienté objet et familier
 - ▶ Pas de concept nouveaux
 - ▶ Pas de pointeurs
 - ▶ POO sans héritage multiple
 - ▶ Syntaxe C
- ▶ Robuste et sûr
 - ▶ Ramasse-miette (Garbage Collector)
 - ▶ Exécution dans machine virtuelle
- ▶ Indépendant de la machine employée pour l'exécution
 - ▶ Programme compilé une fois et exécutable sur plusieurs plateformes
 - ▶ Exécution dans une machine virtuelle spécifique à la plateforme cible

Historique (1)

- ▶ 1990 - le **Projet Stealth**
 - ▶ Alternative à C++ trop complexe pour un projet de « Système embarqué » de Sun
 - ▶ Développement en interne
 - ▶ Renommé **Green Project**
 - ▶ Elaboration d'une technologie pour le développement d'applications de nouvelle génération (opportunité pour SUN)
- ▶ Sept. 1992 - PDA Star7 avec interface graphique
 - ▶ Système Green
 - ▶ Langage Oak
- ▶ Nov. 1992 **Green Project** devient **FirstPerson**
 - ▶ Réponse à un appel d'offre de Time Warner pour construire un décodeur multifonction.
 - ▶ Trop de fonctionnalités proposées, c'est Silicon Graphics qui obtient le marché

Historique (2)

- ▶ Juillet 1994 – recentrage de la plateforme vers le Web
 - ▶ **Oak** devient **Java 1.0** (café)
 - ▶ Le navigateur **Netscape** annonce le support de **Java**
- ▶ 2000 – Java 2
 - ▶ **Java 2 Standard Edition (J2SE)**, plateforme avec les API et bibliothèques de bases, devenue Java SE.
 - ▶ **Java 2 Enterprise Edition (J2EE)**, extension avec des technologies pour le développement d'applications d'entreprise, devenue Java EE.
- ▶ 2022 – Version la plus récente : **Java SE 17**

4. Les Objets

4.1. Présentation de l'Objet

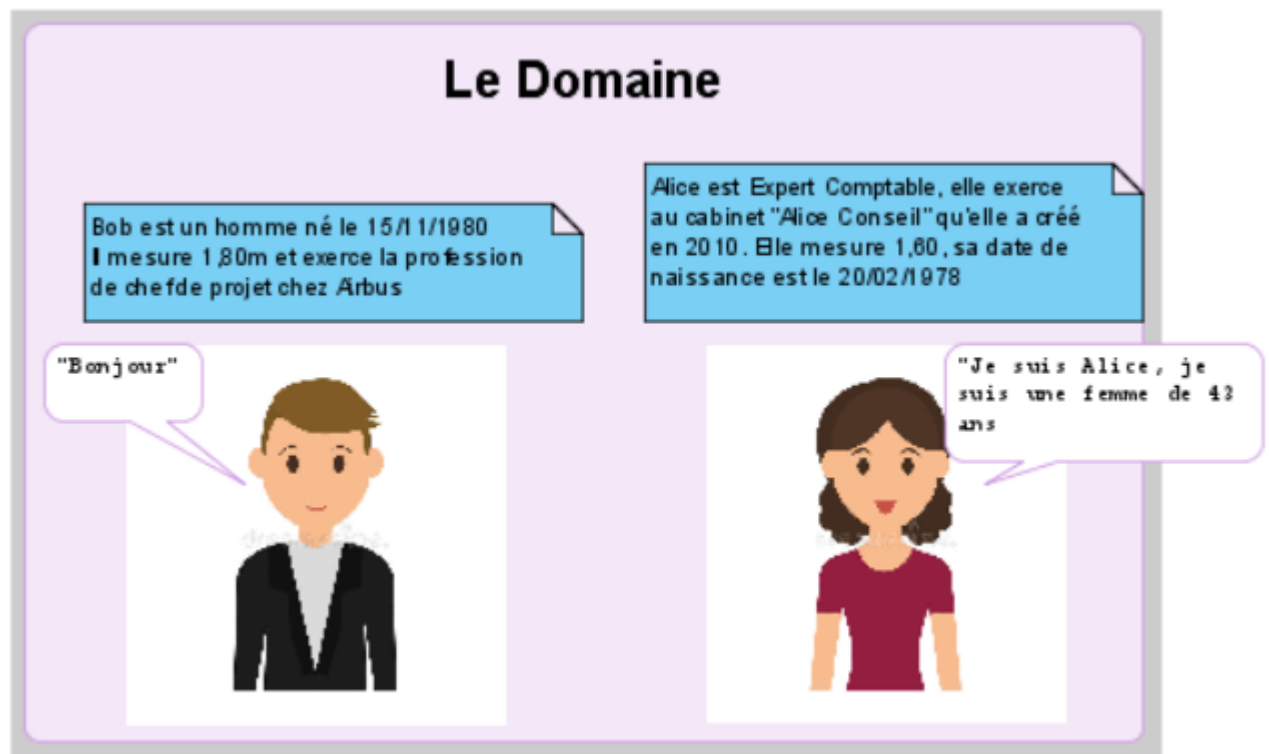
La vidéo du professeur Xavier Blanc de l'université de Bordeaux présente le concept d'objet et la communication entre objets :

<https://www.youtube.com/watch?v=BNEtWb3WceQ&list=PLuNTRFkYD3u5ibkjD1nHP9QZXPjtnPXL>

4.2. Un Exemple

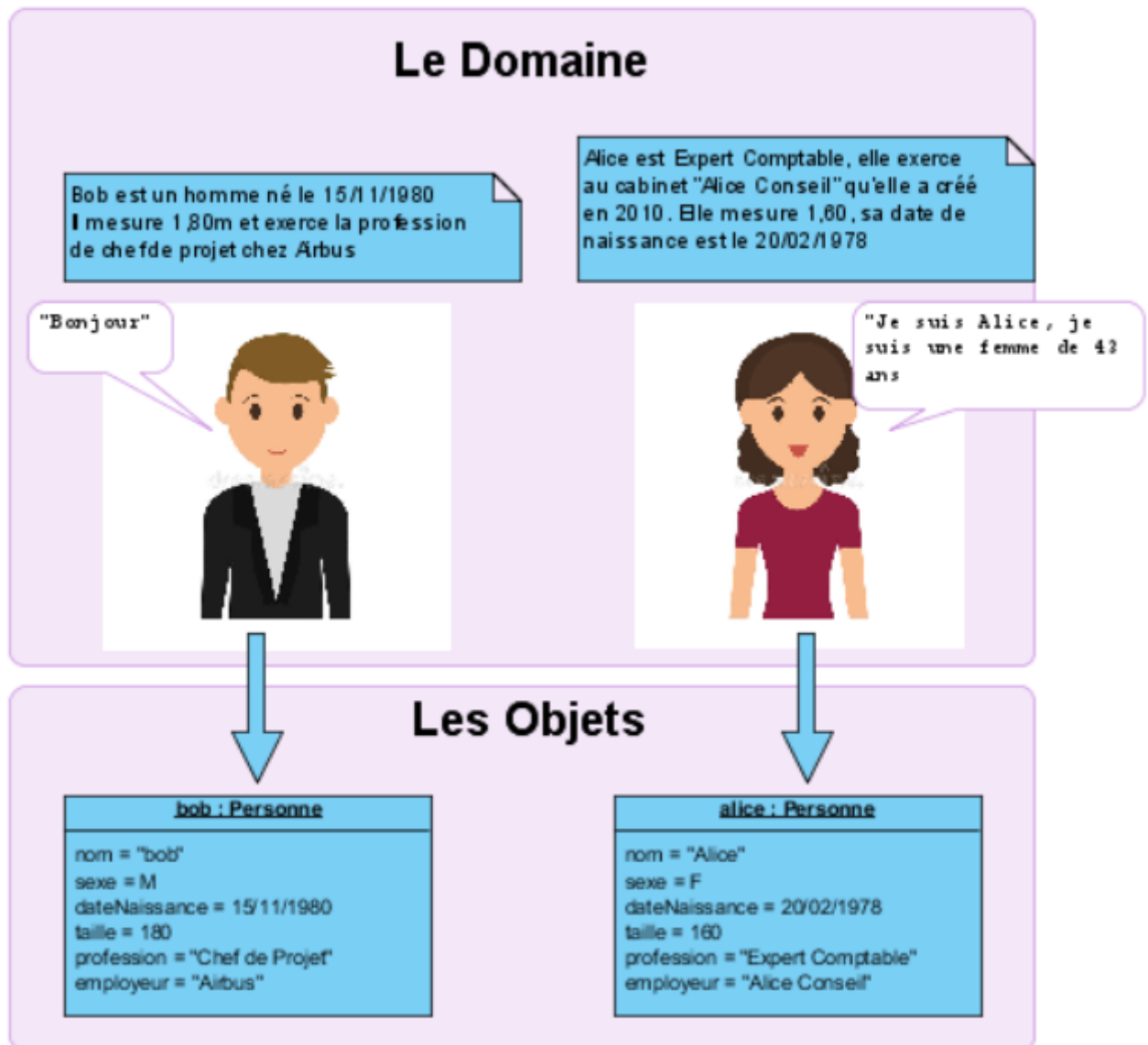
Nous voulons gérer une population de personnes, aujourd'hui notre « cheptel » est composé de 2 personnes mais est appelé à s'agrandir.

4.2.1. Quels sont les objets de ce domaine ?



4.2.2. Les objets du domaine

Les objets de ce domaine sont les personnes « Alice » et « Bob »
Ils comportent chacun les données qui les caractérisent, ce sont leurs propriétés.



Ces objets sont représentés en utilisant le formalisme UML. Il s'agit des objets tels qu'ils sont représentés dans le **Diagramme d'Objets de UML**.

Avons-nous complètement décrit ces objets ?

4.2.3. Le comportement de nos objets **Personne**

Non nous n'avons pas décrit complètement ces objets, les actions que peuvent réaliser les personnes ne sont pas représentées. Elles constituent leurs **Comportements, Opérations** ou bien **Méthodes** et peuvent être invoquées par l'envoi de message à ces objets.

1. Une Personne sait « **Dire Bonjour** » : lorsque l'on envoie le message « direBonjour » à un objet Personne, l'objet répond en retournant un message « 'Bonjour' ».
2. Une Personne sait « **Recevoir un Bonjour** » : lorsque l'on envoie le message « bonjour » à un objet Personne, l'objet apprécie cette salutation mais ne retourne aucun message.
3. Une Personne sait « **Se Présenter** » : lorsque l'on envoie le message « sePresenter » à un objet Personne, l'objet répond en retournant un message « 'Je suis [Son Nom], [un homme ou une femme] de [Son Age] ans' »

Certains comportements peuvent nécessiter des données complémentaires à celles de l'objet, elles seront fournies dans le message utilisé pour l'invocation.

4. Une Personne sait « **Dire Bonjour à quelqu'un par son nom** » : lorsque l'on envoie le message « direBonjourA (unNom) » à un objet Personne, l'objet répond en retournant un message « 'Bonjour [unNom]' ».

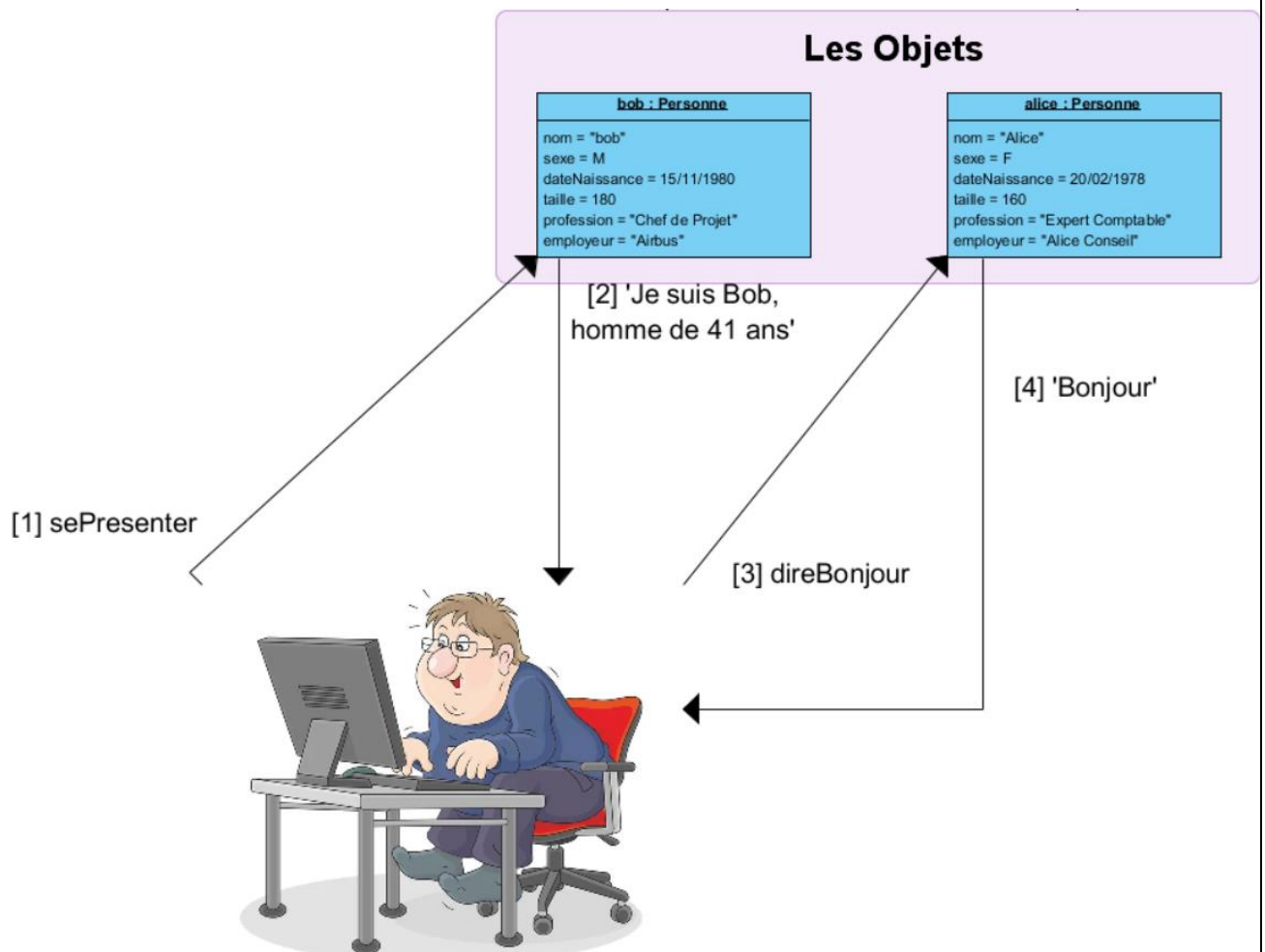
Nos objets **Personnes** auront donc les méthodes suivantes :

- direBonjour(): chaîne
- recevoirBonjour()
- direBonjourA(unNom: chaîne): chaîne
- sePresenter(): chaîne

Ces méthodes seront utilisées par les **Clients** des objets. On appelle **Client** d'un objet tout objet connaissant cet objet et l'utilisant en lui envoyant des messages pour invoquer ses comportements.

L'ensemble des comportements d'un objet qui sont disponibles pour ses **Clients** est appelé **Interface de l'objet**.

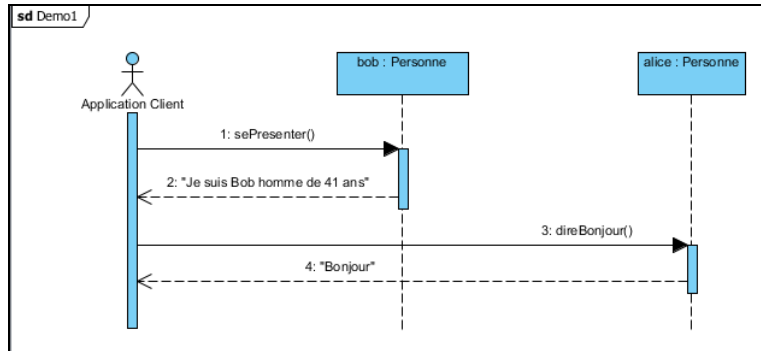
4.2.4. Nos objets **Personne** utilisés par une application



L'application possède (connaît) les objets **bob** et **alice**. Elle envoie des messages à ces objets pour obtenir des comportements et rendre les services attendus par l'utilisateur de l'application.

4.2.5. Les diagrammes d'interactions de UML

Le Diagramme de séquence :



- Les objets sont représentés avec leur nom et leur type.
- Chacun d'eux a une **ligne de vie** qui représente le temps s'écoulant du haut vers le bas.
- L'échange de messages entre objets est matérialisé par des flèches partant de la ligne de vie de l'émetteur vers la ligne de vie du destinataire.
- Les réponses peuvent être représentées par une flèche en pointillé partant de l'émetteur de la réponse vers le destinataire de cette réponse. Lorsque le message d'appel est **synchrone** cette représentation du retour est optionnelle.

5. Classe d'objet

5.1. Objectifs

- Classe, propriété, méthode, objet
- Définition d'une nouvelle classe d'objet
- Représentation des classes en UML
- Codage d'une classe en Java
- Instanciation d'un objet
- Encapsulation

5.1.1. Notion de classe, de propriété, de méthode

Une **classe** est un **moule**, un **patron** à partir duquel des objets pourront être créés.

Une classe regroupe :

- des **propriétés ou attributs** ou données membres,
- des **méthodes** ou fonctions membres.

A partir d'une classe (le modèle), des objets sont créés (instanciés) pour être utilisés.

Chaque objet **connait ses caractéristiques** (les valeurs affectées à ses propriétés) et **sait réaliser des opérations** (ses méthodes).



Créer une **classe**, c'est créer un nouveau **type de données**.

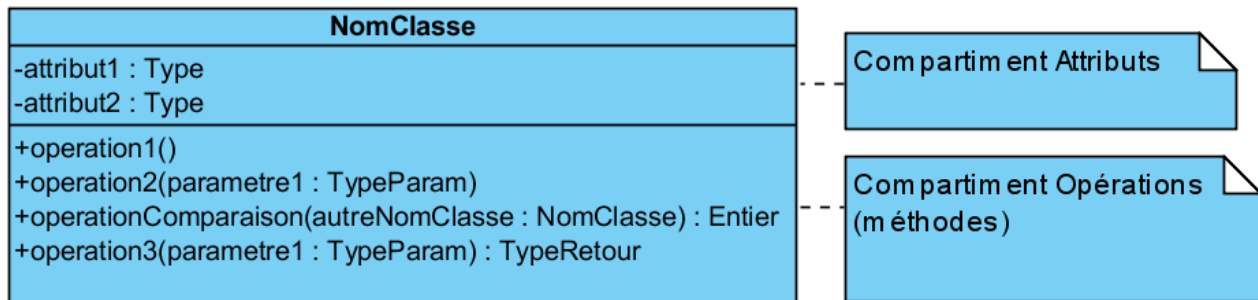
Pour définir la classe d'un objet, il faut donc énumérer toutes les propriétés de cet objet et toutes les fonctions qui vont permettre de définir son comportement. Ces dernières peuvent être classées de la manière suivante :

- Fonction pour lire les valeurs des propriétés d'un objet. (Accesseurs)
- Fonction pour modifier les valeurs des propriétés d'un objet. (Modifieurs)
- Fonctions de calcul
- Opérateurs pour comparer 2 objets (=, >, <)
- Fonctions de conversion vers d'autres types
- Fonctions pour contrôler l'intégrité de l'objet.
- Fonctions d'entrées/sorties pour lire et écrire des données sur les périphériques (clavier, écran, fichier)

5.1.2. Les Classes en UML

Une **classe** est un **moule**, un **patron** à partir duquel des objets pourront être créés.

Représentation des classes en UML :

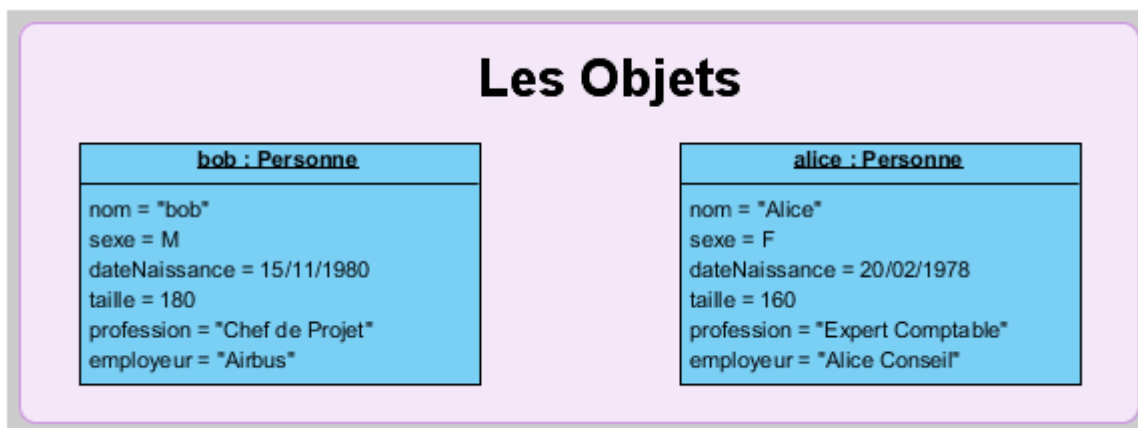


En UML on représente toutes les classes (et leurs relations) dans les diagrammes de classes. Les caractères '-' et '+' devant les attributs et les opérations définissent leur visibilité.

- '-' signifie **Privé** : ne peut être « vu » « lu » et « modifié » que par la classe elle-même. Les clients des objets de cette classe ignorent l'existence des éléments définis comme privés.
- '+' signifie **Public** : est visible et modifiable par tous les clients des objets de cette classe.

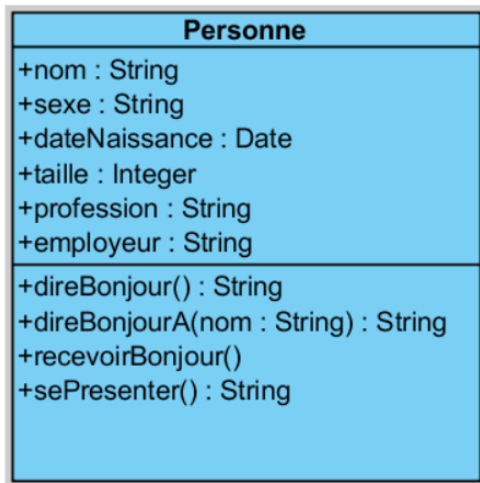
EXERCICE : La classe Personne en UML

Représenter la classe « Personne » permettant de « Classifier » ces objets en UML :



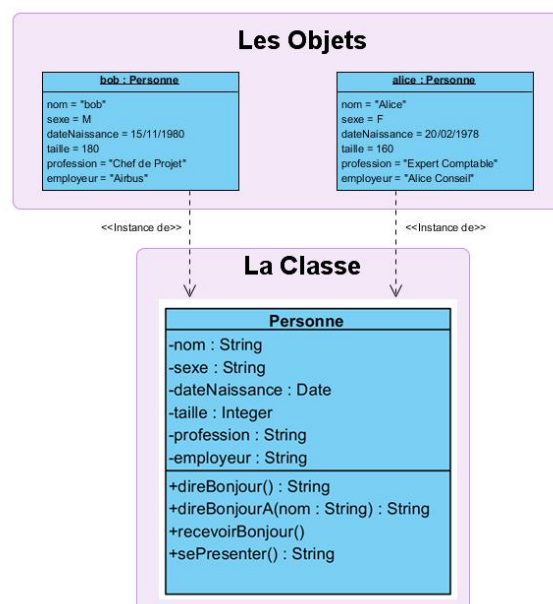
Il faut définir : le nom de la classe, ses attributs et leur type, ses opérations et leur type.

La classe « Personne » :

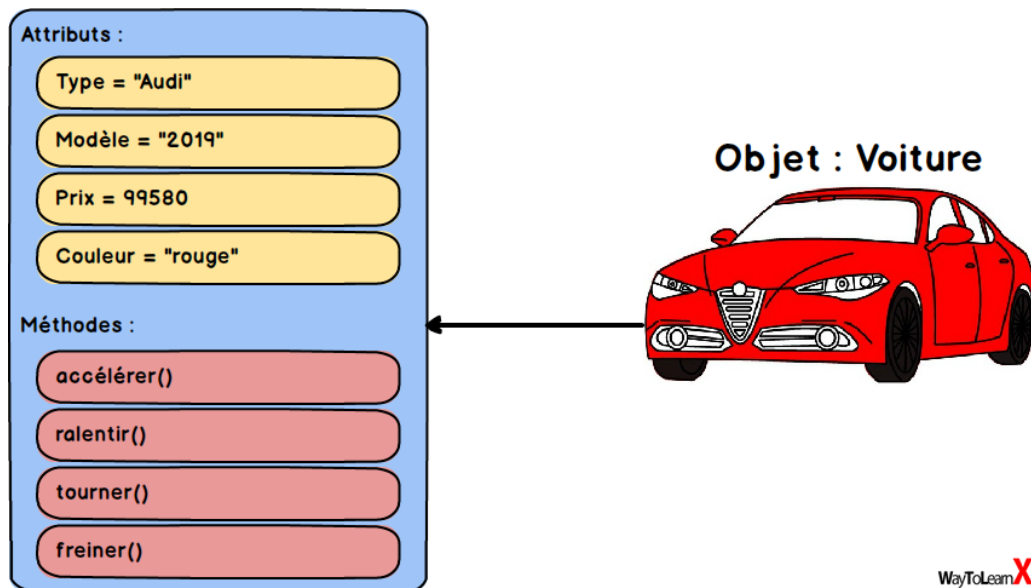


- La classe **est nommée** « Personne »
- Elle possède **6 attributs nommés et typés** :
 - nom, sexe, profession et employeur qui sont des chaînes de caractères
 - dateNaissance qui est du type Date
 - taille qui du type Integer est destinée à recevoir la taille de la personne estimée en cm
- Elle comporte **4 opérations nommées et typées** :
 - Les méthodes direBonjour, direBonjourA et sePresenter sont du type chaîne de caractères car elles retournent une String.
 - La méthode recevoirBonjour ne retourne rien elle du type Void

- La classe « Personne » introduit **un nouveau type** nommé « Personne ».
- bob et alice sont des **objets** du type « Personne »
- bob et alice sont des **instances** de la classe « Personne »
- bob et alice associent une valeur à chacun de leurs **attributs**, ce sont leurs **propriétés**
- bob et alice possèdent les mêmes **comportements** que toutes les « Personne »
- il pourrait maintenant exister **un autre objet** toto qui serait **une autre instance** de la classe « Personne »



La classe « Voiture » :



WayToLearnX

5.1.3. Définition d'une classe en Java

Convention de nommage Java:

<http://www.loribel.com/java/normes/nommage.html>

PascalCase : convention de nommage dans laquelle la première lettre de chaque mot est en majuscule.

Exemple : *MaVoiture* , *AgeDuCapitaine*, *B*

<https://techterms.com/definition/pascalcase>

CamelCase : convention de nommage dans laquelle la première lettre de chaque mot est en majuscule mis à part la première lettre du premier mot qui est en minuscule

Exemple : *maVoiture* , *ageDuCapitaine*, *b*

<https://techterms.com/definition/camelcase>

Convention de nommage :

- Les noms de type sont notés en **PascalCase**. (noms de classes, d'interfaces, d'énumérations)
- Ne pas préfixer les noms des interfaces de la lettre « I »
- Les noms des propriétés et des fonctions sont notés en **CamelCase**
- Ne pas préfixer le nom des propriétés privées du caractère ' _ '

```
// Package dans lequel se trouve la classe  
package edu.cai2022.utc503.poo;
```

```
// Importation des types utilisés  
import java.util.Date;
```

```
public class ClasseJava {  
  
    // Déclaration des propriétés  
    /**  
     * propriete1  
     */  
    public String nomPropriete1;  
  
    /**  
     * propriete2  
     */  
    private Integer nomPropriete2 = 42;  
  
    // Déclaration des méthodes  
  
    /**  
     * Une procédure (ne renvoie pas de résultat)  
     */  
    public void methode1() {  
        // code de la procédure  
        nomPropriete2 = nomPropriete2 + 1;  
    }  
  
    /**  
     * UNE méthode qui retourne  
     * une chaîne de caractère  
     * @return  
     */  
    public String methode2() {  
        return "Résultat en chaîne";  
    }  
  
    /**  
     * Calcule le blablabla... et le retourne  
     * @param param le nombre en paramètre  
     * @return  
     */  
    public Integer methode3(Integer param) {  
        return nomPropriete2 * param;  
    }  
  
    public String methode4(Integer param1, String param2, Date dateCalcul) {  
        return param1 + " " + param2 + " est Le résultat !!!";  
    }  
  
} // Fin de la classe
```

Par convention :

- Les noms de classes (tous les types) commencent toujours par une majuscule (**PascalCase**).
- Les noms des propriétés et des fonctions commencent par une minuscule (**CamelCase**)

Lors de sa déclaration, une propriété peut être initialisée (cf. nomPropriete2).

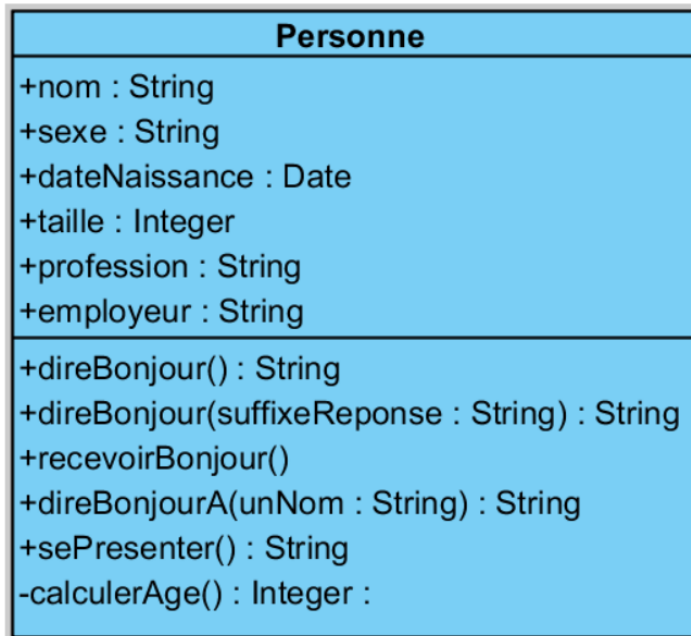
Les méthodes sont implémentées soit par des **procédures** ou par des **fonctions** qui peuvent avoir aucun, un ou plusieurs paramètres.



La **surcharge** d'une procédure ou d'une fonction consiste à définir celle-ci en **plusieurs versions : même nom, même type de retour** mais avec des **paramètres différents**. Une même méthode peut avoir plusieurs **signatures**.

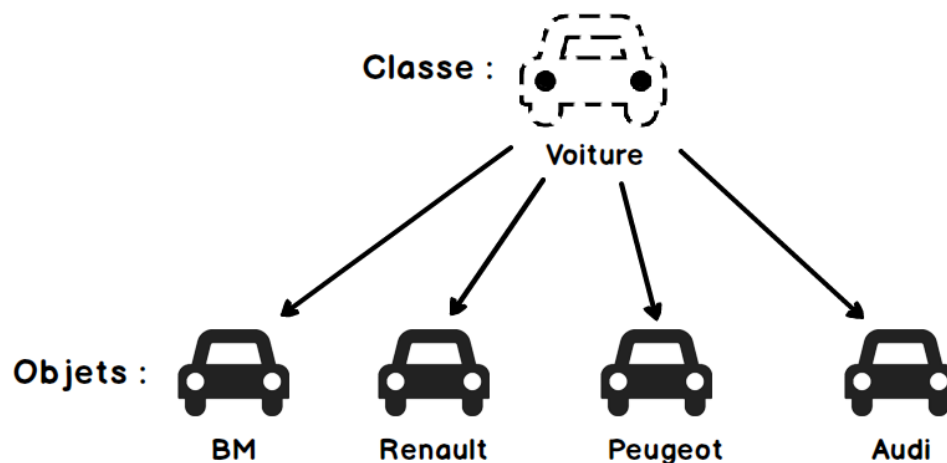
EXERCICE : La classe Personne en Java

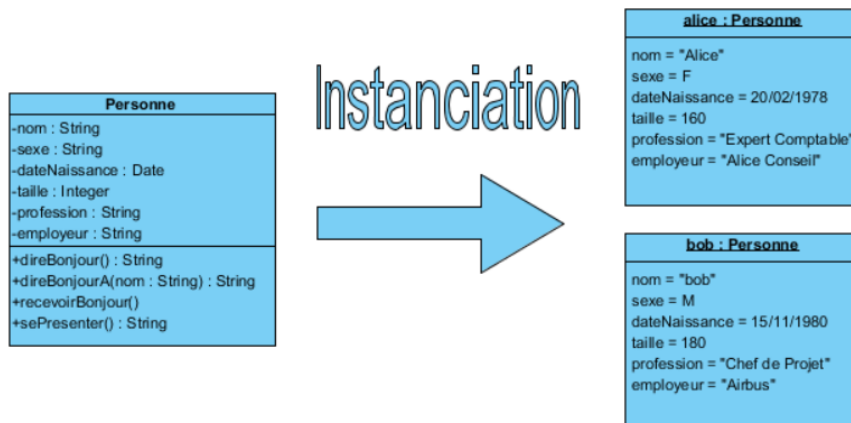
Coder en Java la classe Personne, écrire le code dans un fichier Personne.java dans le package edu.cai2022.utc503.poo

**5.1.4. Instanciation d'une classe**

L'action de créer un objet d'une classe s'appelle l'instanciation. On dit qu'on instancie un objet d'une certaine classe.

Cela correspond dans un langage objet à allouer de la mémoire et y créer une structure de données conforme à la définition de la classe. Cette structure sera pointée par une variable (référence) qui permettra de lire ou écrire les valeurs des propriétés visibles et invoquer les comportements spécifiés par la classe (opérations visibles).





Instantiation d'une classe Java :

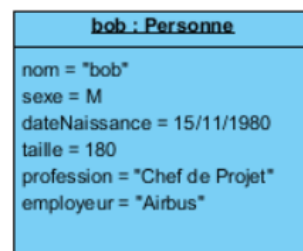
En Java (comme dans d'autres langages objets) l'instanciation d'une classe se fait en utilisant l'opérateur **new**.

```
// Création d'une instance de Personne et affectation de celle-ci à la variable bob
Personne bob = new Personne();
```

Lire et écrire les valeurs des propriétés d'un objet :

L'opérateur «.» permet d'accéder aux propriétés publiques de l'objet pour lire ou écrire leur valeur

```
// Création de l'objet bob de la classe Personne et affectation des valeurs de ses propriétés
Personne bob = new Personne();
bob.nom = "Bob";
bob.sexe = "M";
bob.dateNaissance = new Date("80, 10, 15");
bob.taille = 180;
bob.profession = "Chef de Projet";
bob.employeur = "Airbus";
```



Nous verrons par la suite que le principe d'encapsulation interdit la modification directe des propriétés

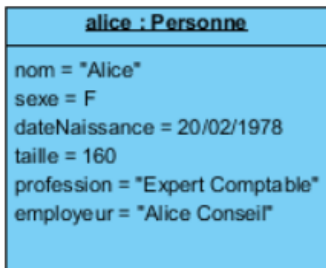
Invoquer les opérations d'un objet :

L'opérateur «.» permet d'accéder aux méthodes disponibles de l'objet pour les invoquer

```
bob.sePresenter()
```

EXERCICE : Instanciation de « Alice »

Ecrire le code Java pour instancier et initialiser l'objet « Alice »



Le Constructeur :

L'instanciation d'une classe se fait en appelant une opération particulière : **le constructeur**

- L'appel au constructeur se fait en utilisant l'opérateur **new**.
- Il existe pour toute classe un **constructeur** par défaut ne prenant aucun paramètre.
- En UML, en Java et dans de nombreux langages orientés objet Il porte le même nom que la classe.
- Son rôle est de créer une instance de la classe et de l'initialiser, pour cela il peut avoir des paramètres. La définition d'un constructeur explicite dans une classe se substitue au constructeur par défaut.
- En POO quand la surcharge est supportée il est possible de créer plusieurs constructeurs avec des paramètres différents.

```
package edu.cai2022.utc503.poo.cours;

import java.util.Date;

/**
 * Classe Personne
 */
public class Personne {

    private String nom;
    private String sexe;
    private Date dateNaissance;
    private Integer taille;
    private String profession;
    private String employeur;

    /**
     * Constructeur de la classe personne
     *
     * @param nom
     * @param sexe
     * @param dateNaissance
     * @param taille
     * @param profession
     */
}
```

```
* @param employeur
*/
public Personne(String nom, String sexe, Date dateNaissance, Integer taille,
                 String profession, String employeur) {
    super();
    this.nom = nom;
    this.sexe = sexe;
    this.dateNaissance = dateNaissance;
    this.taille = taille;
    this.profession = profession;
    this.employeur = employeur;
}
}
```

Après la création de notre constructeur, ce code n'est plus valide (Pourquoi ?) :

```
Personne bob = new Personne();
bob.nom = "Bob";
bob.sexe = "M";
bob.dateNaissance = new Date("80, 10, 15");
bob.taille = 180;
bob.profession = "Chef de Projet";
bob.employeur = "Airbus";
Terminal.ecrireString(bob.sePresenter());
```

bob : Personne
nom = "bob"
sexe = M
dateNaissance = 15/11/1980
taille = 180
profession = "Chef de Projet"
employeur = "Airbus"

Réponse : Parce que notre constructeur vient se substituer au constructeur par défaut, Il ne peut plus être utilisé. Le code correct devient :

```
Personne bob : Personne = new Personne("Bob", "M", new Date(80, 10, 15), 180, "Chef de Projet", "Airbus");
Terminal.ecrireString(bob.sePresenter());
```

EXERCICE : Instanciation de « Alice » (V2)

Adapter le code de l'instanciation d'Alice de l'exercice précédent pour utiliser notre nouveau constructeur.

Notre programme au final :

```
// Programme principal

Personne bob = new Personne("Bob", "M", new Date(80, 10, 15), 180, "Chef de Projet", "Airbus");
Personne alice = new Personne("Alice", "F", new Date(78, 01, 20), 160, "Expert Comptable", "Alice
Conseil");
Personne toto = new Personne("Toto", "M", new Date(60, 06, 14), 163, "Développeur Java", "Sysord");

Terminal.ecrireString(bob.direBonjour());
Terminal.ecrireString(bob.sePresenter());
bob.recevoirBonjour();
Terminal.ecrireString(alice.direBonjourA("Bob"));
```

Une petite modification du programme principal :

```
Personne bob = new Personne("Bob", "M", new Date(80, 10, 15), 180, "Chef de Projet", "Airbus");
Personne alice = new Personne("Alice", "F", new Date(78, 01, 20), 160, "Expert Comptable", "Alice
Conseil");
Personne toto = new Personne("Toto", "M", new Date(60, 06, 14), 163, "Développeur Java", "Sysord");

Terminal.ecrireString(bob.direBonjour());
Terminal.ecrireString(bob.sePresenter());
bob.recevoirBonjour();
Terminal.ecrireString(alice.direBonjourA("Bob"));
toto.dateNaissance = new Date(150, 01, 01);
Terminal.ecrireString(toto.sePresenter());
```

```
Bob: Bonjour
Bob: Je suis Bob un homme de 41 ans
Bob: Bonne journée à vous aussi.
Alice: Bonjour Bob
Toto: Je suis Toto un homme de -28 ans
```

Que s'est-il passé ?
Comment éviter ce type d'erreur ?

- La propriété DateNaissance de toto a été initialisée avec une valeur incorrecte ce qui provoque une erreur de calcul ou du moins un résultat incorrect.
- Il faudrait interdire la modification directe de la valeur des propriétés : les propriétés sont passées en visibilité **privée** ce qui interdit de les modifier.
- Elles sont initialisées lors de l'appel au constructeur, dans ce code il est possible de vérifier la validité des données pour garantir que les opérations ne seront pas en échec à cause de valeurs de propriétés incorrectes.

Nous avons réalisé l'**encapsulation** des propriétés de notre classe.

Personne
-nom : String -sexe : String -dateNaissance : Date -taille : Integer -profession : String -employeur : String
+Personne(nom : String, sexe : String, dateNaissance : Date, taille : Integer, profession : String, employeur : String) +direBonjour() : String +direBonjourA(nom : String) : String +recevoirBonjour() +sePresenter() : String

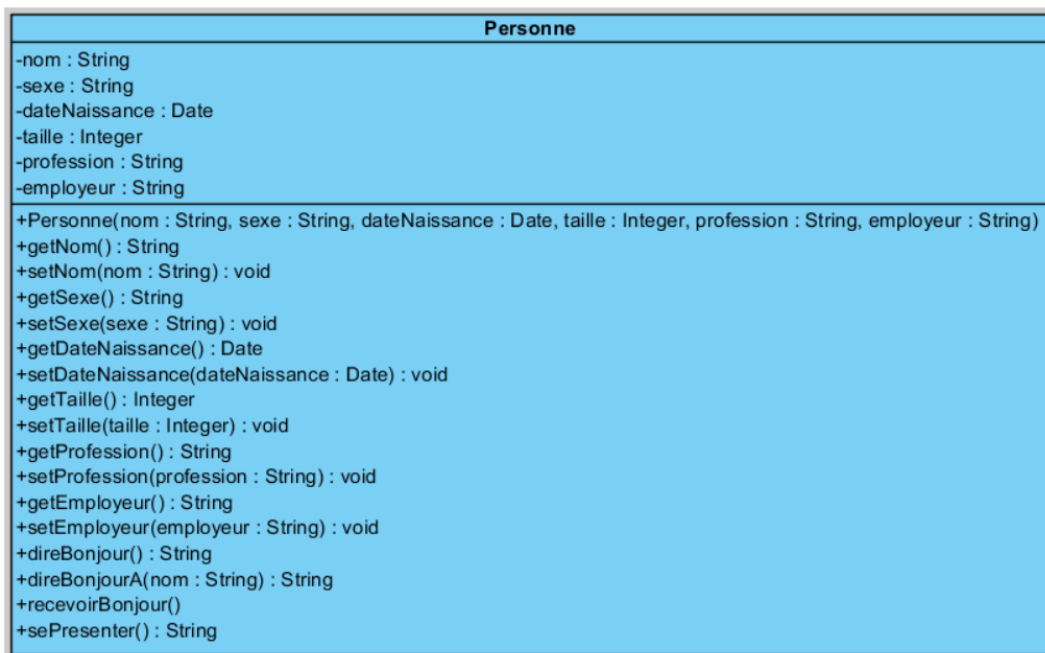
La classe personne est non **mutable** : il n'est pas possible de modifier les propriétés d'un objet de cette classe une fois qu'il est créé. L'ensemble des valeurs des propriétés d'un objet constitue son état. Un objet personne ne changera pas d'état après sa création.

Limites de notre version :

- On ne peut pas modifier les valeurs des propriétés mais dans certains cas cela pourrait être nécessaire. Exemple : Si Alice se marie elle pourrait devoir changer de nom.
- On ne peut pas lire la valeur des propriétés. Elles sont toutes privées.

Pour permettre la lecture et la modification (contrôlée) de nos propriétés nous allons mettre à la disposition des clients de notre classe des **Accesseurs pour la lecture** et des **Modifieurs pour permettre de changer la valeur des propriétés de manière contrôlée**.

Les **accesseurs** sont parfois appelés « **Getter** » et les **modifieurs** « **Setter** » car ces opérations sont nommées **setNomPropriete** et **getNomPropriete**.



Modifieurs et Accesseurs en Java :

Ces opérations spécifiques sont implémentées dans la classe Personne pour chaque propriété.

```
/**
 * Nom de la personne
 */
private String nom;

/**
 * Accesseur à la propriété nom
 * @return
 */
public String getNom() {
    return nom;
}

/**
 * Modifieur de la propriété nom
 * @param nom
 */
public void setNom(String nom) {
    this.nom = nom;
}
```

EXERCICE : modifieurs et accesseurs pour la classe PPersonne

Ajouter les modifieurs et accesseurs pour toutes les propriétés de la classe Personne.

6. Encapsulation – Protection et accès aux données

6.1. Objectifs

- Protection des données
- Méthodes d'accès aux propriétés : **Getter** et **Setter**
- Abstraction procédurale

6.2. Ce qu'il faut savoir

6.2.1. Protection des données

En POO, on évite d'accéder directement aux propriétés par l'opérateur « . » . En effet, cette possibilité ne correspond pas au concept d'encapsulation. En Java, c'est le programmeur qui choisit si une donnée ou une fonction membre est accessible directement ou pas.

On l'indique au moment de la déclaration d'un attribut ou d'une méthode. Pour cela, il existe principalement 3 mots-clés (**modificateurs de visibilité**) :

public	Les données ou fonctions membres publiques sont accessibles à l'extérieur de la classe.
private	Les données ou fonctions membres sont privées à la classe, elles sont accessibles uniquement par les méthodes internes à la classe. L'utilisation du mot private permet de masquer les données ou les fonctions membres, elles ne seront pas accessibles à l'extérieur de la classe.
protected	Les données ou fonctions membres sont privées à la classe mais elles sont néanmoins accessibles dans les classes dérivées (notion d'héritage).

La distinction entre **private** et **protected** n'est visible que dans le cas de la définition de nouvelles classes par héritage. Ce concept est abordé plus loin.

6.2.2. Accesseurs et Modifieurs

Si les propriétés sont **privées**, on ne peut plus y avoir accès directement à l'extérieur de la classe par l'opérateur « . », il faut donc créer des méthodes **publiques** pour accéder aux propriétés qui doivent être visibles à l'extérieur de la classe.

Méthode pour renvoyer la valeur de la propriété : **Get (Accesseur ou Getter)**

```
/**
 * Accesseur au nom
 * @return
 */
public String getNom() {
    return nom;
}
```

Les fonctions Get peuvent être considérées comme un message envoyé à l'objet : "Quelle est la valeur de ta propriété nommée ... ?"

Méthode pour modifier la valeur d'une propriété : **Set (Modifieur ou Setter)**

```
/**
 * Modifieur du nom
 * @param nom
 */
public void setNom(String nom) {
    this.nom = nom;
}
```

Les fonctions Set peuvent être considérées comme un message envoyé à l'objet : *"Maintenant la valeur de ta propriété est ..."*.



Le mot-clé **this**, utilisable à l'intérieur de toute classe, désigne l'instance courante. En Java il est utilisé pour lever les ambiguïtés lorsqu'une variable locale ou un paramètre porte le même nom qu'une propriété de la classe. Dans ce cas pour exprimer de manière explicite que l'on fait référence à la propriété on la désigne par **this.nomPropriété**

Les fonctions accesseurs pourront être utilisées de la manière suivante :

```
Personne bob = new Personne("Bob", Sexe.MASCULIN, new Date(80, 10, 15), 180, "Chef de Projet");
bob.setNom("Tutu"); // modification de la valeur de la propriété nom
Terminal.ecrireString(bob.getNom()); // affichera Tutu
```

L'intérêt de passer par des fonctions **Set** est de pouvoir y localiser des **contrôles de validité** des valeurs passées en paramètres pour assurer la cohérence de l'objet en y déclenchant des **exceptions** par exemple. Nous aborderons ce point dans la suite de ce support.



Si certaines données membres doivent être en **lecture seule**, on ne fournit pas la méthode **set**, ainsi la donnée ne pourra pas être assignée depuis l'extérieur de la classe.

6.2.3. Abstraction Procédurale

Les méthodes publiques exposées par une classe constituent l'interface de cette classe et par extension l'interface des objets de cette classe. Les clients des objets de la classe peuvent invoquer les méthodes en passant les paramètres attendus et ils obtiendront le résultat attendu.

L'objet peut être vu comme une **boîte noire** on sait ce qu'il fait mais pas comment il le fait.

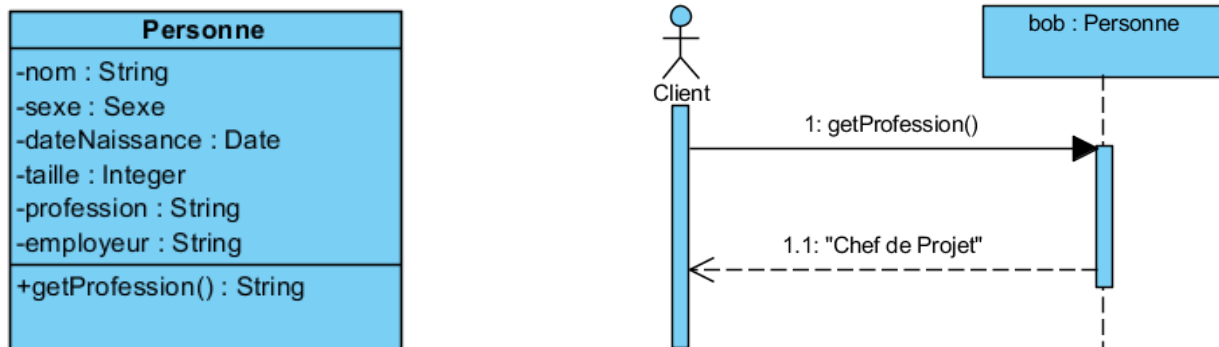
Toute invocation de méthode d'un objet est **atomique** (appel → résultat ou bien erreur) du point de vue du client.

Cet effet **boîte noire** et **invocation atomique** est obtenu par l'encapsulation.

Un exemple :

Cas initial :

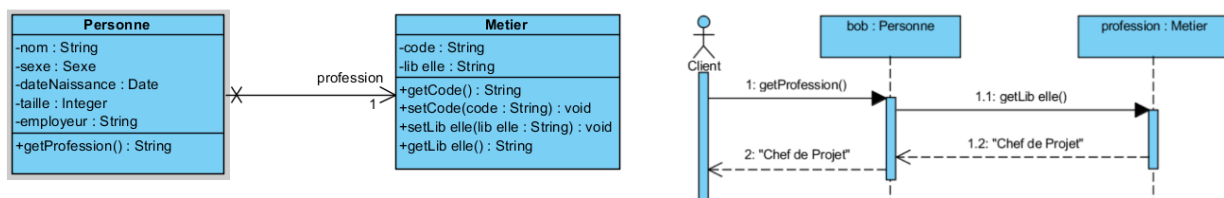
La classe *Personne* a un propriété « profession » de type *string*. Le client qui invoque la méthode *getProfession()* obtient le nom de la profession de bob, la valeur de la propriété « profession » de l'objet bob.



Evolution :

Pour éviter les redondances, la classe « *Personne* » n'a plus sa propriété « profession » de type *string*, elle a été remplacée par une association vers la classe « *Metier* » avec comme rôle « profession ».

Le client qui invoque la méthode *getProfession()* obtient le nom de la profession de bob, la valeur de la propriété « libelle » de la « profession » de bob.



Grace à l'encapsulation, le client n'est pas impacté par l'évolution.

6.2.4. Encapsulation

L'**encapsulation** est un concept de la POO¹ qui permet de rassembler les propriétés et les méthodes d'un objet pour les manipuler dans une seule entité appelée classe d'objet. Le code des méthodes et la définition des propriétés sont enfermés dans la classe qui est une sorte de **boîte noire**.

Les applications clientes ne connaissent que l'**interface** (visibilité Publique) des objets qu'elles utilisent et n'ont pas à se soucier du fonctionnement interne de la classe.

Conséquences :

- Les clients ne connaissent pas les propriétés présentes dans la classe car elles sont privées.

¹ POO = Programmation Orientée Objet

- Si le code de la classe est modifié mais pas l'interface (même nom de méthode, mêmes arguments), les applications clientes ne seront pas modifiées. Il est possible de modifier le code d'une méthode ou sa stratégie de réalisation sans que les clients ne s'en rendent compte.

6.2.5. Types énumérés

Un type qui propose une liste fixée de valeurs possibles peut être représenté par une énumération de ses valeurs. On dira qu'il s'agit d'un type énuméré ou une énumération.

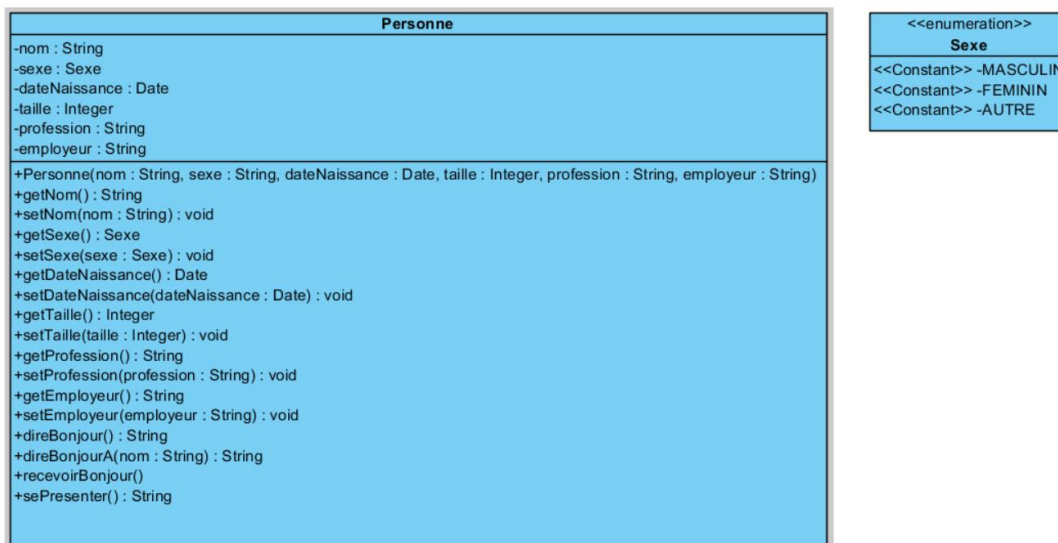
Exemples :

La réponse à une question qui peut avoir comme valeur: « *Oui* », « *Non* », « *Ne se prononce pas* »
Un état pour une tâche : « *Planifiée* », « *En Cours* », « *Terminée* »

Pour notre Personne nous pouvons utiliser une énumération pour définir le sexe : « Masculin », « Féminin ».

Représentation en UML :

Les stéréotypes « *enumeration* » et « *Constant* » nous permettent de définir une énumération et ses valeurs. Notre propriété sexe est maintenant du type Sexe et plus du type String.



En Java :

```

/**
 * Type énumérée Sexe
 */
public enum Sexe {
    MASCULIN,
    FEMININ
}

/**
 * Classe Personne

```

```
*/
public class Personne {

    /**
     * Sexe de la personne
     */
    private Sexe sexe;

    // ...

    /**
     * Constructeur de la classe personne
     *
     * @param nom
     * @param sexe
     * @param dateNaissance
     * @param taille
     * @param profession
     * @param employeur
     */
    public Personne(String nom, Sexe sexe, Date dateNaissance, Integer taille, String
        profession, String employeur) {
        this.nom = nom;
        this.sexe = sexe;
        this.dateNaissance = dateNaissance;
        this.taille = taille;
        this.profession = profession;
        this.employeur = employeur;
    }

    public Sexe getSexe() {
        return sexe;
    }

    public void setSexe(Sexe sexe) {
        this.sexe = sexe;
    }

    // ...
}
```

EXERCICE : Les sociétés

1. A partir de ces fiches descriptives des sociétés, proposer un diagramme d'objet UML comportant ces 3 entreprises
2. Ajouter une nouvelle société à votre diagramme d'objet.
3. A partir de votre diagramme d'objet, déduire et proposer une classe « Société » que vous représenterez dans un diagramme de classes UML.
4. Coder votre classe en Java avec son constructeur, ses Modifieurs et ses Accesseurs. Dans le code, vérifier la validité des valeurs avant de les affecter aux propriétés (exemple: le numéro siren ne doit comporter que des chiffres et faire exactement 10 caractères)
5. Ajouter la méthode obtenirIdentite() qui retourne une chaine de présentation de la société (nom, localisation, « En activité depuis ... » ou « Activité suspendue depuis ... »).
6. Ajouter une méthode suspendreActivite() qui passera l'entreprise de l'état « Active » à « Suspendue » et une méthode reprendreActivite() qui passera de l'état « Suspendue » à l'état « Active ».
7. Pour valider votre implémentation, créer une classe de test comprenant une méthode main, instanciez au moins 2 sociétés et appelez les différentes méthodes pour les tester.

Entreprise Sysord :

Renseignements juridiques

Date création entreprise	14-04-2006 - il y a 15 ans	Statuts constitutifs >
Forme juridique	SARL unipersonnelle	Voir (RCS) >
Noms commerciaux	SYSORD	
Téléphone	 Afficher le numéro de téléphone	
Adresse postale	19 AV HIPPOLYTE PUECH 12250 TOURNEMIRE	
Numéros d'identification		
Numéro SIREN	489803569 	
Numéro SIRET (siège)	48980356900032 	
Numéro TVA Intracommunautaire	FR11489803569 	
Numéro RCS	Rodez B 489 803 569	
Informations commerciales		
Catégorie	Ingénierie	
Activité (Code NAF ou APE)	Ingénierie, études techniques (7112B)	Voir (RCS) >
Informations juridiques		
Statut RCS	✓ INSCRITE - au greffe de Rodez	 Extrait d'immatriculation RCS
Statut INSEE	✓ INSCRITE	 Avis de situation SIRENE
Date d'immatriculation RCS	Immatriculée au RCS le 02-05-2006	
Date d'enregistrement INSEE	Enregistrée à l'INSEE le 14-04-2006	
Taille de l'entreprise		
Effectif (tranche INSEE à 18 mois)	Unité non employeuse ou effectif inconnu au 31/12	Voir (RCS) >
Capital social	20 000,00 €	Voir (RCS) >

Airbus

Date création entreprise	18-10-1991 - <i>Il y a 30 ans</i>	Statuts constitutifs >
Forme juridique	Société par actions simplifiée	Voir (PLUS) >
Noms commerciaux	AIRBUS	
Téléphone	Afficher le numéro de téléphone	
Adresse postale	2 RPT EMILE DEWOITINE 31700 BLAGNAC	
Numéros d'identification		
Número SIREN	383474814 	
Número SIRET (siège)	38347481400100 	
Número TVA Intracommunautaire	FR89383474814 	
Número RCS	Toulouse B 383 474 814	
Informations commerciales		
Catégorie	Autres industrie	
Activité (Code NAF ou APE)	Construction aéronautique et spatiale (3030Z)	Voir (PLUS) >
Informations juridiques		
Statut RCS	✓ INSCRITE - au greffe de Toulouse	Extrait d'immatriculation RCS
Statut INSEE	✓ INSCRITE	Avis de situation SIRENE
Date d'immatriculation RCS	Immatriculée au RCS le 04-11-1991	
Date d'enregistrement INSEE	Enregistrée à l'INSEE le 18-10-1991	
Taille de l'entreprise		
Effectif moyen	9 160	Voir (PLUS) >
Capital social	3 576 769,00 €	Voir (PLUS) >
Chiffre d'affaires 2020	39 036 834 000,00 €	

Alice Conseil

Renseignements juridiques		
Date création entreprise	01-01-2015 - <i>Il y a 7 ans</i>	Statuts constitutifs >
Forme juridique	SARL unipersonnelle	Voir (PLUS) >
Noms commerciaux	ALICE SENLIS CONSEIL	
Téléphone	Afficher le numéro de téléphone	
Adresse postale	270 AV DE L ESPACE 59118 WAMBRECHIES	
Numéros d'identification		
Número SIREN	808424980 	
Número SIRET (siège)	80842498000023 	
Número TVA Intracommunautaire	FR54808424980 	
Número RCS	Lille Metropole B 808 424 980	
Informations commerciales		
Catégorie	Services	
Activité (Code NAF ou APE)	Conseil pour les affaires et autres conseils de gestion (7022Z)	Voir (PLUS) >
Informations juridiques		
Statut RCS	✓ INSCRITE - au greffe de Lille Metropole	Extrait d'immatriculation RCS
Statut INSEE	✓ INSCRITE	Avis de situation SIRENE
Date d'immatriculation RCS	Immatriculée au RCS le 22-12-2014	
Date d'enregistrement INSEE	Enregistrée à l'INSEE le 01-01-2015	
Taille de l'entreprise		
Capital social	5 000,00 €	Voir (PLUS) >

La classe Société (à compléter) :

Societe
-siren : String
-formeJuridique : FormeJuridique
-raisonSociale : String
-dateCreation : Date
-telephone : String
-capital : Integer

<<enumeration>> FormeJuridique
<<Constant>> -SAS
<<Constant>> -SARL
<<Constant>> -EURL
<<Constant>> -SA

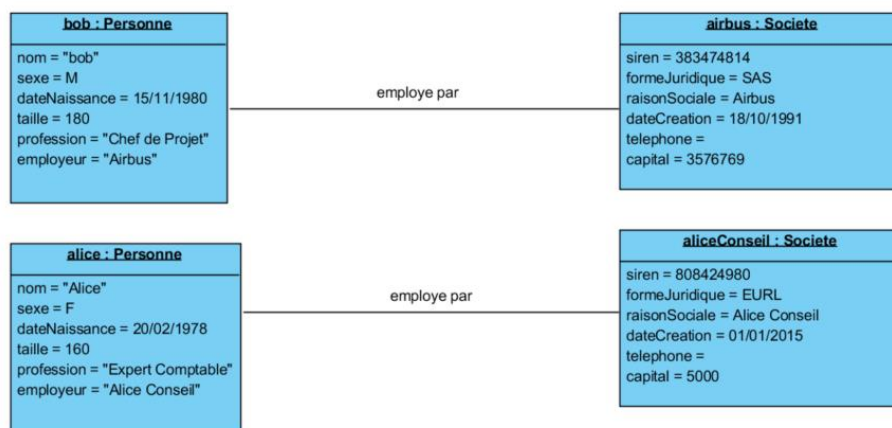
7. Les liens entre objets

7.1. Le contexte

Nous avons créé dans les exercices précédents les classes « Personne » et « Société ». Nous souhaitons maintenant utiliser ces deux classes pour modéliser le fait qu'une personne est employée par une société.

7.2. Représentation dans le diagramme d'objet UML

Nous avons dans le diagramme d'objet représenté les instances des personnes Bob et Alice et des sociétés Airbus et Alice Conseil. Pour représenter que Bob est employé par Airbus et que Alice est employée par Alice Conseil il faut créer un lien entre chacune de ces personnes et la société qui les emploie.

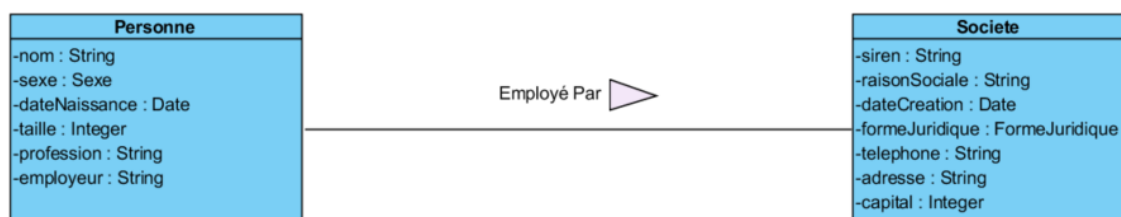


Ce lien représenté par ce trait entre la personne et la société qui l'emploie signifie que l'instance de « Personne » connaît l'instance de « Société » qui l'emploie et peut donc lui envoyer des messages.

7.3. Représentation dans le diagramme de classes UML

7.3.1. Associations

Dans le diagramme de classe on modélise le fait qu'il existera un lien entre les instances de classes par ce que l'on appelle une « **Association** ». Une association est représentée par un trait entre les classes participantes à l'association. L'association est nommée pour permettre d'en comprendre la signification mais ce nom n'est pas obligatoire.

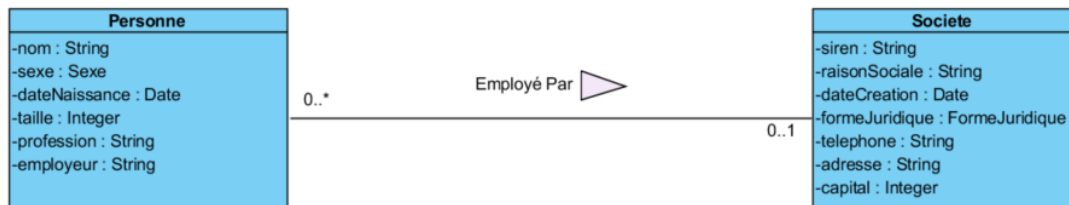


Si le nom de l'association est ambiguë et ne permet pas de comprendre le sens de lecture on peut ajouter un triangle dont la pointe montre le sens de lecture.

Ce diagramme avec cette association nous fournit-il toutes les informations nécessaires à l'implémentation (traduction en code) ?

7.3.1. Multiplicités

L'association permet de représenter le lien entre classes, par contre le nombre d'instances qui peuvent participer à l'association n'est pas précisé. Cette information qui précise le nombre d'occurrences s'appelle la « **Multiplicité** »



La multiplicité représente le nombre d'instances d'une classe qui peuvent participer à l'association. Cette information se place sur le diagramme du côté de la classe dont on veut préciser le nombre d'occurrence. Cette information est obligatoire

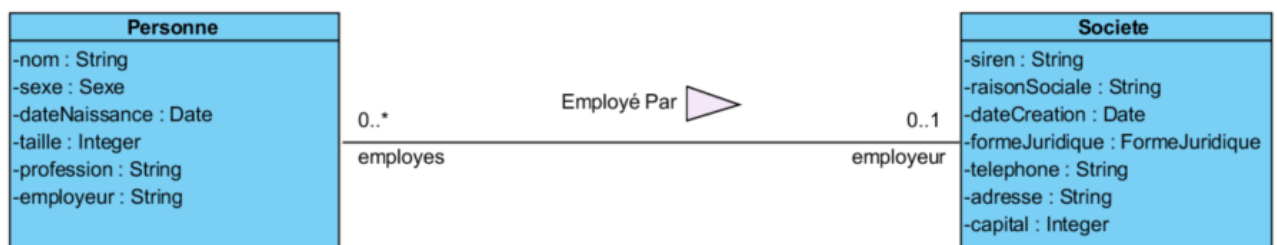
Signification :

0..1 du côté Société : Une personne est employée au minimum par aucune Société et au maximum une seule société.

0..* du côté Personne : Une société emploie au minimum aucune Personne et au maximum N personne. (N : toute valeur entière positive)

Rôles

Une autre information complémentaire obligatoire pour une association est le rôle que joue chacune des classes dans le cadre de cette association. Ce rôle permet de nommer le lien d'une instance vers une ou plusieurs autres.



Dans cette association « Employé Par » la Société joue le rôle de l'**employeur** et les personnes sont les **employes**.

- Une personne connaît la société pour laquelle elle travaille éventuellement sous le nom de « **employeur** ».
- Une société connaît toutes les personnes qu'elle emploie sous le nom de « **employes** ».

Différents types d'associations :

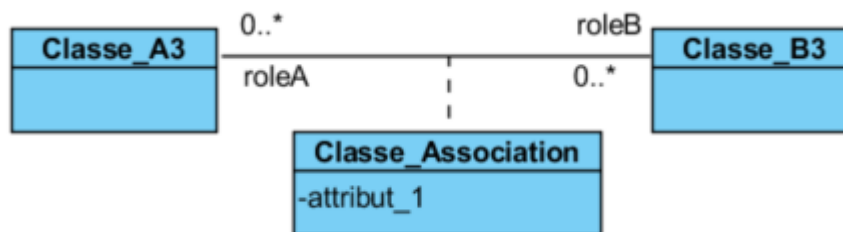
Association « One To Many » ou de droite à gauche « Many To One »



Association « Many To Many »



Association « Many To Many » avec une classe d'association. Il existe des informations contextuelles à l'association qui ne peuvent être affecté ni à la classe A3 ni à la classe B3, ces informations sont pour chaque occurrence de l'association (pour une instance de A3 et une instance de B3).



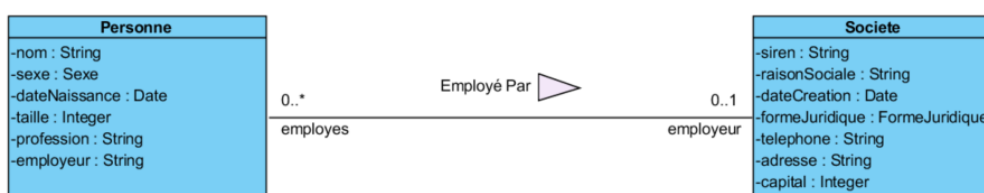
7.3.2. Associations Unidirectionnelles vs Bidirectionnelles

La personne connaît son employeur, le côté du rôle employeur de l'association est dit **Navigable**. La société connaît ses employés, le côté du rôle « employés » est dit **Navigable**. Une association dont les deux côtés sont navigables est dite **Bidirectionnelle**, si un seul des côtés est navigable elle est dite **Unidirectionnelle**.

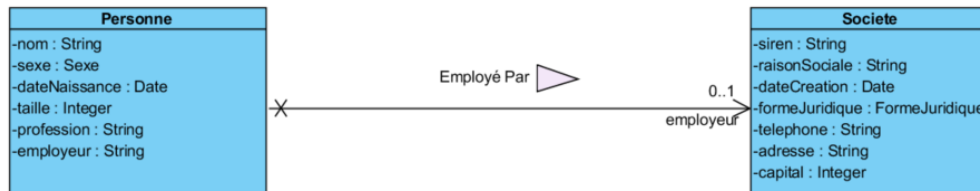
L'implémentation d'une association **Bidirectionnelle** impose la synchronisation des liens dans les classes participantes :

L'affectation d'une **Société** en tant qu' « employeur » d'une **Personne** implique que cette Personne appartienne aux « employés » de la société.

Association bidirectionnelle :

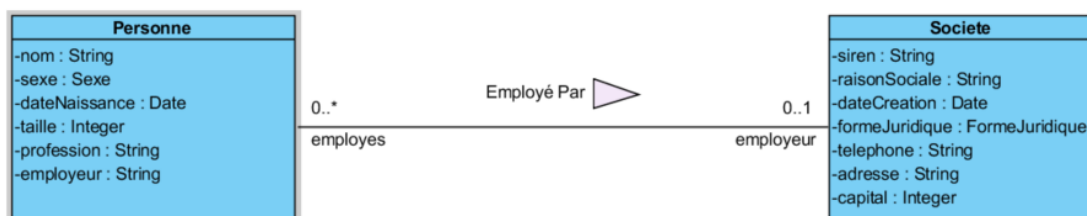


Association unidirectionnelle (la société ne connaît pas ses employés) :
Pour montrer qu'une association n'est pas navigable dans un sens on marque ce côté de l'association avec une croix.



7.4. Implémentation des associations

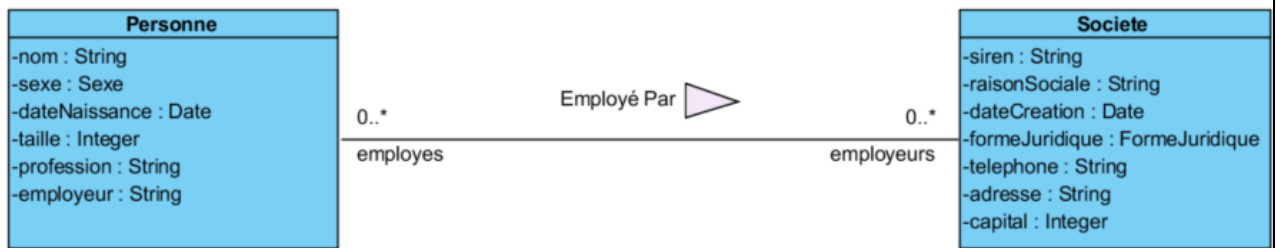
7.4.1. Associations One To Many ou Many To One



Une Personne peut être employée ou non par une Société
Une Société emploie de zéro à plusieurs Personnes

<pre> /** * Classe Personne * @class */ class Personne { //... /** * Employeur de la personne */ private Societe employeur; public Societe getEmployeur() { return employeur; } public void setEmployeur(Societe employeur) { this.employeur = employeur; } //... } </pre>	<pre> /** * Classe Societe * @class */ class Societe { //... /** * Employés de la société */ private List<Personne> employes = new ArrayList<>(); public boolean addEmploye(Personne personne) { return employes.add(personne); } public boolean removeEmploye(Personne personne) { return employes.remove(personne); } //... } </pre>
--	--

7.4.2. Associations Many To Many



Une Personne peut être employée par aucune ou plusieurs Sociétés
 Une Société emploie de zéro à plusieurs Personnes

```

/**
 * Classe Personne
 * @class
 */
class Personne {
    //...

    /**
     * Les Employeurs de la personne
     */
    private List<Societe> employeurs = new ArrayList<>();

    public boolean addEmployeur(Societe e) {
        return employeurs.add(employeur);
    }

    public boolean removeEmployeur(Societe e) {
        return employeurs.remove(employeur);
    }

    //...
}
    
```

```

/**
 * Classe Societe
 * @class
 */
class Societe {
    //...

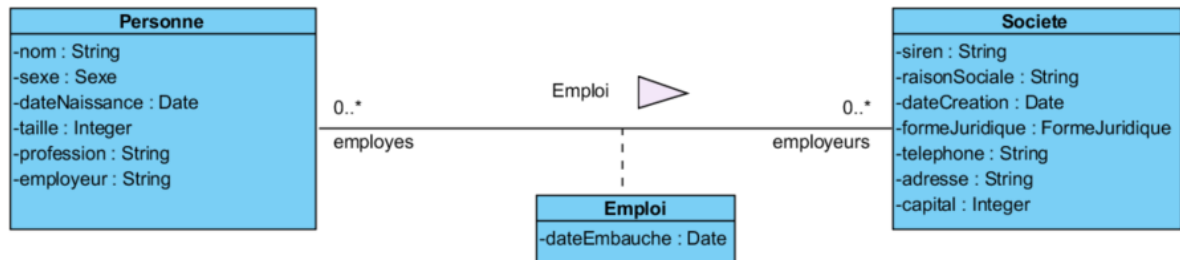
    /**
     * Employés de la société
     */
    private List<Personne> employes = new ArrayList<>();

    public boolean addEmploye(Personne personne) {
        return employes.add(personne);
    }

    public boolean removeEmploye(Personne personne) {
        return employes.remove(personne);
    }

    //...
}
    
```

7.4.3. Associations Many To Many avec une classe d'association



```

public class Employi {
    private Societe employeur;
    private Personne employe;
    private Date dateEmbauche;

    public Employi(Societe employeur, Personne employe, Date dateEmbauche) {
        super();
        this.employeur = employeur;
        this.employe = employe;
        this.dateEmbauche = dateEmbauche;
    }
}

```

```

/**
 * Classe Personne
 * @class
 */
class Personne {
    //...
    /**
     * Les emplois de la personne
     */
    private List<Employi> emplois = new ArrayList<>();
    //...
}

```

```

/**
 * Classe Societe
 * @class
 */
class Societe {
    //...
    /**
     * Les emplois de la société
     */
    private List<Employi> emplois = new ArrayList<>();
    //...
}

```

Conclusion :

- Lorsqu'il existe des liens entre objets, ils sont représentés dans le diagramme d'objets par un trait entre les instances.
- Dans le diagramme de classes ces relations sont des associations entre les classes.
- Ces associations comportent des Rôles, Multiplicités et peuvent être Bidirectionnelles ou bien Unidirectionnelles
- Les associations Bidirectionnelles doivent être synchronisées ce qui rend leur implémentation en est plus complexe que pour les associations Unidirectionnelles
- La sémantique d'une association entre des classes A et B peut être l'une de celles-ci :
 - **B Appartient ou appartiennent à A**
 - **A Contient un ou plusieurs B**
 - **A utilise un ou plusieurs B**

EXERCICE : le jeu de cartes

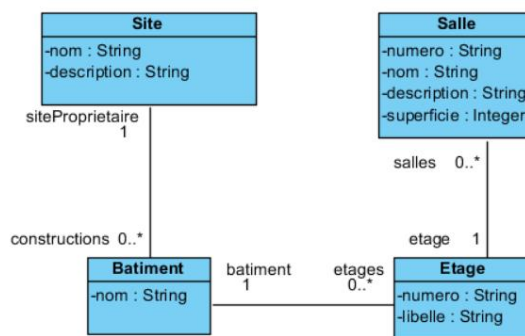
Un jeu de carte est composé d'un ensemble de cartes ayant chacune d'elle une Couleur (Pique, Trèfle, Carreau, Cœur) et une Valeur (Nombre, Roi, Dame etc.). Selon leur utilisation, un jeu de carte peut comporter un ensemble de cartes différent en nombre (32, 54, autre) ou en cartes (avec ou sans Joker).

Un jeu de carte peut être « Coupé » : le jeu comportant toutes les cartes du jeu est séparé en deux sous-paquets, le sous-paquet du dessus est placé sous le paquet du dessus pour reconstituer le jeu.

Un jeu de carte peut être « Mélangé » : l'ordre d'un certain nombre de cartes du jeu est modifié aléatoirement.

1. Proposer un diagramme de classes pour modéliser un jeu de cartes.
2. Coder en Java les classes du diagramme réalisé à la question 1.
3. Utiliser les classes Java du jeu de carte pour coder le jeu de la bataille entre un utilisateur et l'ordinateur.

EXERCICE : la visite guidée d'un site



4. A partir de ce diagramme de classes, coder ces classes en Java et instancier un site ressemblant à celui de 2ISA (créer de nouvelles classes si elles vous paraissent nécessaires)
5. Créer une classe Guide qui permettra de guider l'utilisateur dans sa visite le site. L'utilisateur sera placé au début de la visite dans un site.
 - a. A partir d'un site : il sera proposé à l'utilisateur la liste des bâtiments du site pour qu'il choisisse celui dans lequel il veut rentrer.
 - b. Lorsque l'utilisateur est dans un bâtiment (dans le hall) : il lui sera possible de sortir du bâtiment et retourner dans le site ou de choisir de se rendre à un étage particulier
 - c. Lorsque l'utilisateur est à un étage (dans le couloir de l'étage) : il lui sera possible de retourner dans le hall du bâtiment ou de choisir de rentrer dans une salle de l'étage.
 - d. Lorsque l'utilisateur est dans une salle : il lui sera possible de retourner à l'étage (dans le couloir) ou de demander au guide de lui décrire la salle (afficher la description).

Ce petit logiciel que vous aurez programmé pourrait servir de base à la création d'un jeu d'aventure textuel ou d'un « Escape Game ».

8. Notions diverses de POO

8.1. Objectifs

- Données et méthodes de classe
- Les objets sont des données comme les autres
- Le destructeur

8.2. Propriétés et méthodes d'instance vs de classe

En POO mais aussi dans la plupart des langages de programmation tous les éléments ont une **portée (Scope)**. La portée définit l'espace dans lequel l'élément est connu et accessible.

Portée	Description
Globale	fonction ou donnée accessible à partir de partout
Classe	fonction ou donnée accessible à partir de la classe (pas besoin d'une instance) ou des instances de la classe. Dans de nombreux langages, les éléments de portée Classe sont déclarés avec le mot-clé « static ».
Instance	méthodes ou propriétés d'une classe
Locale	variable déclarée dans un bloc (fonction, structure de contrôle) et accessible uniquement dans ce bloc. (paramètre d'une fonction, variable de travail déclarée dans le code)

8.2.1. Propriété de classe

Jusqu'ici les propriétés qui ont été déclarées sont des **propriétés d'instances**. C'est-à-dire que **chaque objet**, instancié à partir de la même classe, possède **sa propre valeur** pour chaque propriété.

Supposons maintenant que l'on souhaite compter le nombre d'instances créées pour la classe *Personne*. Il semblerait judicieux que le constructeur puisse incrémenter un compteur (la même variable partagée et accessible par toutes les instances) à chaque fois qu'il est appelé. Ce compteur n'est pas propre à une instance mais a rapport avec toutes les instances de la classe. Il aurait donc toute sa place au niveau de la classe.

Il faut donc définir dans la classe *Personne*, une **propriété de classe**, c'est-à-dire une propriété qui possède **une seule valeur** qui est **partagée entre toutes les instances de la classe**.

Une **propriété de classe** est déclarée par le mot-clé **static**.

```
/**
 * Classe Personne
 */
public class Personne {

    /**
     * le compteur du nombre de personnes (portée classe)
     */
    private static Integer nbPersonnes = 0;

    public static Integer getNbPersonnes() {
        return Personne.nbPersonnes;
    }
}
```

```

public Personne(nom : string, sexe : Sexe, dateNaissance : Date, taille : number,
    profession : string) {

    // Incrémentation du compteur d'instances
    Personne.propNbPersonnes++;

    this.nom = nom;
    this.sexe = sexe;
    this.dateNaissance = dateNaissance;
    this.taille = taille;
    this.profession = profession;
}
}

Personne bob = new Personne("Bob", Sexe.MASCULIN, new Date(80, 10, 15), 180, "Chef de Projet")
Personne alice = new Personne("Alice", Sexe.FEMININ, new Date(78, 01, 20), 160, "Expert Comptable");
Terminal.ecrireInt(Personne.nbPersonnes); // affichera 2

```

8.2.2. Méthode de classe

Comme pour les propriétés de classes, il peut exister des méthodes de portée classe. Ces méthodes ne peuvent donc pas accéder aux propriétés de portée instance mais seulement aux propriétés de portée classe.

Une bibliothèque de fonctions sans état (n'utilise pas d'autres données que ses paramètres) est implémentée par un ensemble de méthodes de portée classe.

Exemple :

```

/**
 * Bibliothèque pour manipuler les Personnes
 * @class
 */
public class PersonneBib {
    /**
     * Renomme la personne et la retourne
     */
    public static Personne rebaptiser(Personne personne, String nouveauNom) {
        personne.nom = nouveauNom;
        return personne;
    }
    /**
     * Retourne la personne la plus âgée entre personne1 et personne2
     */
    public static Personne laPlusAgee(Personne personne1, Personne personne2) {
        return personne1.age() > personne2.age() ? personne1 : personne2;
    }
}

```

Le mot-clé **static** indique qu'il s'agit de méthodes **de classe** et non d'instance.



L'appel d'une propriété ou d'une méthode de classe diffère : en effet ce n'est pas à une instance particulière que le message est envoyé mais à la classe elle-même. En d'autres termes, il n'est pas nécessaire d'instancier la classe pour avoir accès à une méthode de classe.

8.3. Les objets sont des données comme les autres

Tout est dans le titre...

Il est possible de passer à des méthodes des objets en tant que paramètre. Les méthodes peuvent retourner des objets. En opposition aux types primitifs qui sont **transmis par valeur**, les objets sont passés par **référence** (par adresse).

Une classe peut avoir des propriétés de type objet, ces propriétés peuvent représenter les liens entre classes déduits des associations.

8.4. Les destructeurs

Le **destructeur** est une méthode qui est appelée quand un objet est détruit (plus utilisé). La destruction intervient **automatiquement** lorsqu'un objet est hors de portée, qu'il n'est plus référencé ou en fin de programme. C'est le **Garbage Collector** ou le ramasse-miettes qui se charge de repérer les objets candidats à la destruction et de les détruire ensuite, il libère ainsi la place mémoire associée.

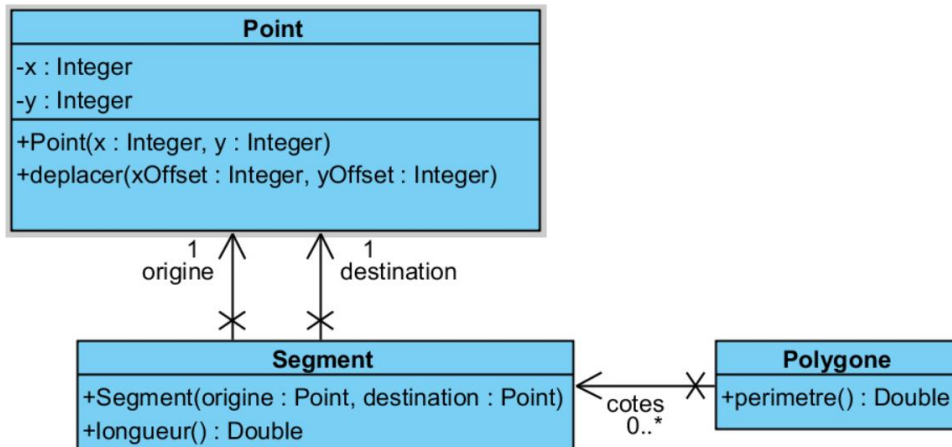
Si un objet devient inutile, il est possible pour le programmeur d'indiquer qu'il est candidat à la destruction en forçant son déréférencement. La destruction de l'objet n'est pas forcément synchrone (immédiat). Dans une classe, on déclare un destructeur uniquement si on a besoin d'écrire un traitement spécifique à déclencher à la destruction de l'objet.

Appeler le destructeur dans le code ne provoque pas la destruction de l'objet. C'est uniquement le ramasse-miettes qui peut l'appeler avant de détruire l'objet.

En Java il existe un ramasse-miette qui collectera les instances qui ne sont plus référencées. La Méthode **finalize()** de l'objet sera appelée à ce moment-là pour exécuter un code particulier à la destruction de l'objet. La plupart du temps, le rôle du destructeur est de libérer les ressources allouées par l'objet.

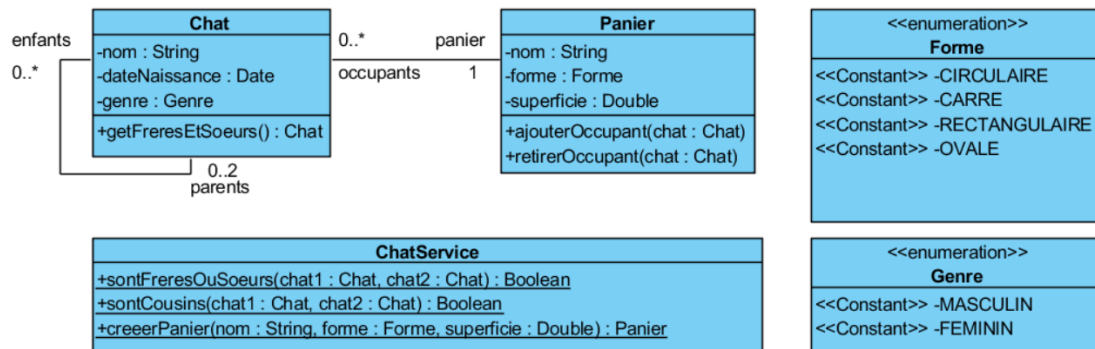
EXERCICE : Point, Segment et Polygone

1. Coder en Java les classes de ce diagramme (propriétés, associations et comportement)
2. Créer un petit programme de démonstration du fonctionnement de votre implémentation.



EXERCICE : Les paniers de chats

1. Coder en Java les classes de ce diagramme (propriétés, associations et comportement)
2. Créer une fonction qui recherche si des chats cousins occupent un même panier.



9. Héritage

9.1. Objectifs

- Héritage
- Classe Object

9.2. Ce qu'il faut savoir

9.2.1. L'héritage

L'héritage est un des principes fondamentaux de la POO. L'héritage consiste en la création d'une nouvelle classe dite **classe dérivée** ou **sous-classe** ou **classe fille** à partir d'une classe existante appelée **super-classe** ou **classe de base** ou **classe mère**. On parle aussi de **généralisation-spécialisation**.

Vidéo du professeur Xavier Blanc :

<https://www.youtube.com/watch?v=bLge8v-czPg&list=PLuNTRFkYD3u5ibkjD1nHP9QZXPjtvnPXL&index=5>

L'héritage permet de :

- **Récupérer le comportement** standard d'une classe et de réutiliser les propriétés et les méthodes définies dans celle-ci.
- **Ajouter des fonctionnalités** supplémentaires en créant de nouvelles propriétés et de nouvelles méthodes dans la classe dérivée.
- **Modifier le comportement** standard d'une classe en remplaçant certaines méthodes de la classe parente dans la classe dérivée (redéfinition).

Apports de l'héritage :

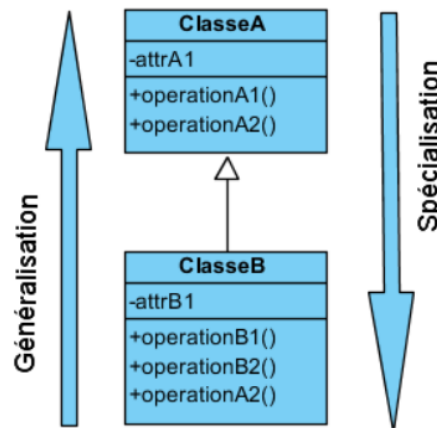
- Il permet la réutilisation de comportement ou concept défini à un niveau général
- Le programmeur dispose d'une bibliothèque d'objets standards qu'il peut adapter pour les besoins de son application.

Certains langages autorisent l'héritage multiple en permettant de créer une classe **reprenant les propriétés et comportements de plusieurs classes mères** mais ce n'est pas sans introduire des problèmes, pour cette raison, la plupart des langages objet ne l'autorise pas. Une classe hérite directement **au plus d'une autre, pas d'héritage multiple**. Java n'autorise pas l'héritage multiple.

Dans de nombreux langages objet, on peut interdire d'hériter d'une classe ou de redéfinir la méthode d'une classe dans ses sous-classes. En C# le mot-clé **sealed** permet de le faire, en Java c'est **final**.

9.2.2. Représentation de l'héritage en UML :

En UML cet héritage entre classe est représenté uniquement dans le diagramme de classe. Le formalisme pour représenter l'héritage entre 2 classes est une flèche particulière allant de la sous-classe vers la super classe.

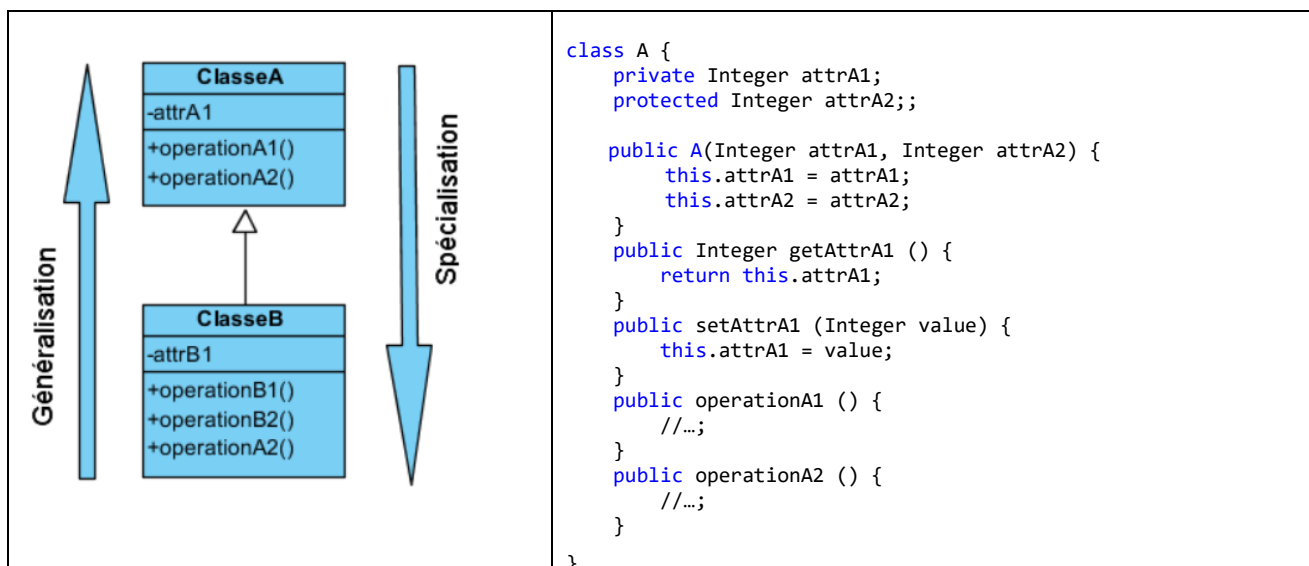


- La Classe B **hérite** de la classe A
- La Classe A est la **super classe**, **classe mère** ou **classe de base** de la classe B
- La Classe A est une **généralisation** de la classe B : tout ce qui est dans B est dans A
- La classe B est une **spécialisation** de la classe A : B comporte tout ce qui est dans A (*operationA1*, *operationA2*) plus ce qui lui est spécifique (*operationB1*, *operationB2*). Certains comportements de A peuvent être **redéfinis** dans B (*operationA2*) pour être **spécialisé**.

En résumé :

- Tout ce que sait faire A, B sait le faire de la même manière (*operationA1*) ou d'une manière adaptée pour correspondre à ce qu'est B (*operationA2*).
- B sait faire des choses que A ne sait pas faire (*operationB1*, *operationB2*)
- B est un A et donc ne peut pas supprimer de comportement issu de A.

9.2.3. Implémentation de l'héritage en Java :



```

class B extends A{
    private String attrB1;

    public B(Integer attrA1, Integer attrA2, String attrB1) {
        // appel constructeur de la super classe
        super(attrA1, attrA2) ;
        this.attrB1 = attrB1;
    }
    public String getAttrB1 () {
        return this.attrB1;
    }
    public setAttrB1 (String value) {
        this.attrB1 = value;
    }
    public operationB1 () {
        // accès direct possible à attrA2 car déclaré avec
        // modifieur protected
        this.attrA2 += 1 ;
        //...;
    }
    public operationB2 () {
        //...;
    }
    public operationA2 () {
        // redéfinition comportement de A dans B;
    }
}

```

9.2.4. Exemple basé sur « Personne » :

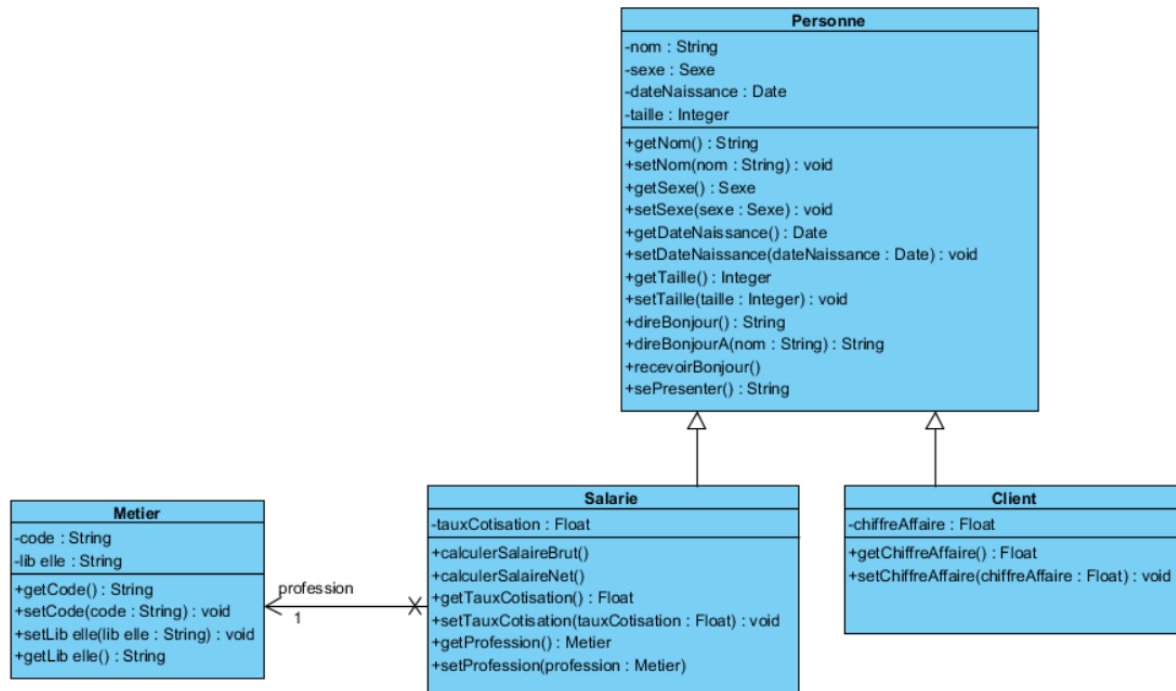
Nous allons réutiliser notre classe **Personne** en tant que super classe et en faire hériter deux sous-classes :

- Un Salarié de notre entreprise **est une** **Personne** qui a un **salaire** que nous devons **calculer**.
- Un Client **est une** **Personne** avec laquelle l'entreprise réalise un **chiffre d'affaire**.

Lorsque la définition d'un type d'objet du domaine inclut « **Est un** » en citant un autre objet du domaine, cela indique qu'il y a une relation **d'héritage (spécialisation)** entre les classes correspondantes.

- Un « Salarié » est une « Personne » particulière à laquelle l'entreprise verse un salaire.
- Un « Salarié » est une spécialisation de « Personne »
- Une « Personne » est une généralisation de « Salarié »
- Un « Client » est une « Personne » particulière avec laquelle l'entreprise réalise un CA.
- Un « Client » est une spécialisation de « Personne »
- Un « Personne » est une généralisation de « Client »

Diagramme de classe UML :



La classe Salarie :

- La classe « Salarie » hérite de la classe « Personne » car un salarié est une personne spécifique qui a une profession et un taux de cotisation utilisé pour calculer son salaire.
- Les propriétés de la classe « Personne » sont héritées mais ne sont pas accessibles car elles ont comme visibilité **privé** qui n'autorise que la classe elle-même à y accéder.
- Toutes les opérations de la classe « Personne » sont héritées et sont accessibles car elles ont comme visibilité **public**.
- La propriété « `tauxCotisation` » utilisée pour calculer le salaire net à partir du salaire brut est une propriété additionnelle de la classe « Salarie » pour assurer sa spécificité.
- L'association « `profession` » est une propriété additionnelle de la classe « Salarie » pour assurer sa spécificité.
- Les opérations `calculerSalaireBrut()` et `calculerSalaireNet()` sont des opérations supplémentaires de la classe « Salarie » pour assurer sa spécificité.

La classe client :

- La classe « Client » hérite de la classe « Personne » car un client est une personne spécifique avec laquelle l'entreprise réalise un chiffre d'affaire.
- Les propriétés de la classe « Personne » sont héritées mais ne sont pas accessibles car elles ont comme visibilité **privé** qui n'autorise que la classe elle-même à y accéder.
- Toutes les opérations de la classe « Personne » sont héritées et sont accessibles car elles ont comme visibilité **public**.
- La propriété « `chiffreAffaire` » utilisée pour mémoriser le chiffre d'affaire réalisé avec le client est une propriété additionnelle de la classe « Salarie » pour prendre en compte sa spécificité.

9.2.5. Visibilité des propriétés et des méthodes dans la classe Fille

En plus des **modificateurs** de visibilité **public** et **private**, il est possible d'utiliser un niveau intermédiaire : **protected**.

Rappelons que :

- Les propriétés ou les méthodes privées d'une classe ne sont accessibles qu'à l'intérieur de cette classe donc pas dans les classes dérivées.
- Les propriétés ou les méthodes publiques d'une classe sont accessibles partout.
- Les propriétés ou les méthodes déclarées avec le mot-clé **protected** sont accessibles dans les classes héritées.

9.2.6. Constructeurs dans une classe dérivée (sous classe)

Un « Salarie » est une « Personne ». Pour créer une instance de « Salarie », il faut d'abord créer la partie « Personne » de l'instance « Salarie » et l'initialiser, puis ensuite l'étendre en y ajoutant la partie « Salarie » et l'initialiser.

En conséquence, le constructeur de la classe dérivée doit obligatoirement appeler le constructeur de la classe Parente.

super([arguments attendus par le constructeur]) appelle le constructeur de la classe Parente en lui passant en paramètres les valeurs nécessaires.

D'une manière générale, toute référence à la super classe est effectuée avec le mot-clé **super**

En résumé, le constructeur d'une classe dérivée :

- Appelle le constructeur de la classe de base en lui passant tous les paramètres dont il a besoin
- Initialise ensuite les propriétés spécifiques de sa classe.

9.2.7. Redéfinition des méthodes de la classe Parente

Les méthodes de la classe de base qui ont été déclarées peuvent être redéfinies dans la classe Dérivée. Pour redéfinir une méthode dans la classe dérivée, il faut conserver la même signature. Ainsi la méthode appelée sera différente selon que l'instance est un objet de la super classe ou bien de la sous-classe (principe induisant le Polymorphisme).

Exemple :

```
/**
 * Classe Personne
 * @class
 */
export class Personne {
    // ...

    /**
     * la personne retourne une chaîne pour se présenter
     * @returns la présentation de la personne
     */
    public String sePresenter()
    {
        return this.nom + ": Je suis " + this.nom + " "
            + this.hommeOuFemme()
            + " de " + this.age() + " ans "
        ;
    }
    // ...
}

/**
 * Classe Salarie
 * @class
 */
export class Salarie extends Personne {
    // ...

    /**
     * la personne retourne une chaîne pour se présenter
     * @returns la présentation de la personne
     */
    public String sePresenter()
    {
        return super.sePresenter() + " ma profession est " + this.profession.getLibelle();
    }
    // ...
}
```

Notez que l'on peut encore faire appel à la méthode de la classe Parente. `super.Methode()` permet de réutiliser le code de la classe de base.

9.2.8. Type statique vs Type dynamique :

Dans les langages fortement typés :

- Un **type** est **compatible** avec **lui-même** et **tous ses sous-types** (classes héritées)
- Toute variable est **déclarée** comme étant d'un **type précis**.
- Une **variable d'un type ne peut recevoir que des valeurs de types compatibles**.
- Lorsque l'on affecte la valeur d'une variable à une autre variable il faut donc que le type de la variable affectée soit compatible avec le type de la variable à affecter.

Le **type statique** d'une variable est le **type** utilisé lors de sa **déclaration dans le code**.

Le **type dynamique** d'une variable est le **type réel de sa valeur**.

Exemple :

```
Personne marie;  
marie = new Salarie("Marie", Sexe.FEMININ, new Date(82, 03, 08), 175, 22);  
Terminal.ecrireStringln(marie.sePresenter());
```

Le **type statique** de « marie » est « Personne »

Le **type dynamique** de « marie » est « Salarie »

Lorsque l'on invoque la méthode `sePresenter()` sur `marie` la méthode à exécuter est dépendante du type dynamique de `marie`. Faut-il appeler la version qui est dans « Personne » ou bien celle qui est dans « Salarie ».

Cette décision ne peut être prise par le compilateur car il ne connaît que les types statiques.

Un mécanisme permettant de faire ce choix est donc nécessaire à l'exécution, c'est le « **late binding** » ou « **Liaison tardive** ».

Late Binding ou Liaison tardive :

De façon générale, si `O` est un objet et `M` une méthode, pour exécuter la méthode `O.M()`, le système cherche la version de la méthode `M` la plus proche du type dynamique de `O` en remontant son arbre d'héritage.

L'environnement d'exécution applique cet algorithme :

- Il recherche la méthode dans le **type dynamique**, si elle est présente il l'exécute Sinon :
- Il recherche la méthode dans la super-classe, si elle est présente il l'exécute Sinon il continue la recherche en inspectant la super classe de cette super classe etc.

9.2.9. Compatibilité et affectation, casting

Dans les langages fortement typés un contrôle des types statiques est effectué à la compilation.

Quelles sont les règles à respecter lors des affectations ?

```
Personne marie;
marie = new Salarie("Marie", Sexe.FEMININ, new Date(82, 03, 08), 175, 22);
Salarie unSalarie = marie; // NE COMPILER PAS
```

unSalarie a pour type Salarie mais marie est déclarée en tant que Personne. Le compilateur lève une erreur car une Personne n'est pas un Salarie. Pourtant marie est bien un Salarie mais le compilateur l'ignore.

Il est possible de forcer le compilateur à accepter cette affectation par une déclaration explicite du type affecté. Cela s'appelle **caster ou transtyper une variable**.

```
Personne marie;
marie = new Salarie("Marie", Sexe.FEMININ, new Date(82, 03, 08), 175, 22);
Salarie unSalarie = (Salarie)marie; // COMPILER CAR CAST EXPLICITE
```

Si la conversion vers le type cible est impossible : **le type destination n'est pas une sous classe du type d'origine**. Une erreur sera levée à la compilation.

Exemple :

```
Personne unePersonne = (Personne) new Metier("M1", "Metier1") as Personne; // NE COMPILER PAS
```

Si la conversion vers le type cible est possible théoriquement mais qu'à l'exécution le type d'origine n'est pas compatible avec celui dans lequel il doit être casté, une erreur de Cast se produira à l'exécution.

Exemple :

```
Client unClient = new Client("Bernard", Sexe.MASCULIN, new Date(77, 08, 08), 190, 2000);
Personne unePersonne = unClient;
Salarie unSalarie = (Salarie) unePersonne; // PRODUIRA UNE ERREUR A L'EXECUTION A L'UTILISATION DE L'INSTANCE
```

Par contre, l'affectation d'une valeur d'une sous-classe à une variable du type de la super classe est possible sans caster car le sous-type est aussi un super type.

Exemple :

```
Personne unePersonne = unSalarie;
```

Cette affectation est acceptée car tout salarié est une personne.

L'affectation `varClasseMere = varClasseFille` est autorisée car une instance de la classe Fille est aussi une instance de la classe Parente. Il n'y a pas de conversion, l'objet référencé par la variable de type `ClasseMere` reste une instance de la classe Fille.

L'affectation inverse `varClasseFille = varClasseMere` génère une erreur de compilation. Mais si la variable `varClasseMere` référence une instance de la classe `ClasseFille` alors on peut effectuer une conversion vers la `ClasseFille`:

Il est possible de forcer la conversion en faisant un **casting** :

```
classeFille = (ClasseFille) classeMere;
```



L'instruction ci-dessus peut provoquer une erreur à l'exécution si la conversion n'est pas possible (si `varClasseMere` n'est pas une instance compatible de `ClasseFille`).

EXEMPLE :

```
// Tout salarié est une personne
// - On peut affecter un Salarié à une variable de type Personne
Personne p = bob;
// - On ne peut pas être sûr qu'une personne est un salarié
// - il faut le dire explicitement en faisant un Cast
Salarie s = (Salarie)p;
// Affectation impossible
// Salarie sa = (Salarie)metierCarreleur;
// Erreur humaine de cast incorrect
Personne p2 = unClient;
Salarie s2 = (Salarie) p2;
Terminal.ecrire(s2.getTauxCotisation());
```

9.2.10. Classe de base

Dans de nombreux langages objet comme Java ou C#, la super classe de tous les objets est Object. **C'est le type racine de toutes les classes.**

9.2.11.Conclusion :

- L'héritage garantit que les classes dérivées d'une classe Parente possèdent toutes les méthodes de cette classe. En d'autres termes, la **superclasse** définit un **comportement commun** pour ses **sous-classes**.
- Centraliser du code commun dans une superclasse permet à toutes les sous-classes d'en hériter : pour changer ce comportement, il suffit de modifier la superclasse et toutes les sous-classes verront le changement.



Attention à ne pas surutiliser l'héritage. L'héritage doit être utilisé pour traduire une relation « **est une sorte de** » entre une sous-classe et une superclasse.

Exercice 14 : Le parc de véhicule

Une entreprise possède un parc de véhicules regroupant : des voitures, des camions, des hélicoptères, des bateaux et des avions.

Chacun de ses véhicules a une immatriculation qui lui est propre ainsi qu'une couleur. Selon le type de véhicule des informations additionnelles permettent de le décrire plus précisément.

Ainsi les bateaux ont un indice de flottaison, les avions un coefficient d'aérodynamisme, les voitures un nombre de places et les camions un nombre d'essieux et un poids.

Le but est de créer un système de classes permettant de classer hiérarchiquement le plus possible les types de véhicule.

Partant des catégories à modéliser, il apparaît assez évident de dériver les classes Voiture et Camion d'une même classe « VéhiculeRoulant » et de laisser de côté les classes Hélicoptère et les Bateaux. Bien que différents, tous les véhicules partagent certaines caractéristiques. Aussi, toutes nos classes vont partager un ancêtre commun que nous appellerons « Véhicule ».

1. Proposer un diagramme de classe UML pour permettre de représenter tous les types de véhicules du parc.
2. Proposer une implémentation en Java

Le Polymorphisme

9.3. Objectifs

- Comprendre le polymorphisme d'héritage
- Méthodes virtuelles

9.4. Ce qu'il faut savoir

Le **polymorphisme** est le concept le plus puissant de l'orienté objet :

- La méthode d'une classe sera **invocable sur toutes ses sous-classes (puisque héritée)**
- La méthode peut **adopter une forme (un comportement) différente** selon la sous-classe.
- Le polymorphisme est obtenu par **redéfinition de la méthode dans les sous-classes** (adaptation du comportement)
- Le polymorphisme est réalisé dans les langages objet par le **principe de la liaison dynamique ou liaison tardive** (late binding).

Dans le chapitre précédent, nous avons vu qu'il est possible d'écrire :

```
Personne p1 = new Salarie("Marie", Sexe.FEMININ, new Date(82, 04, 08"), 175, 22);
```

La référence est du type `Personne`, l'objet est du type `Salarie`. Une telle affectation est correcte et ne génère pas d'erreur de compilation car tout objet qui satisfait au test "*Est une Personne*" peut être affecté à une référence de type `Personne`.

Que se passe-t-il quand le message `sePresenter()` est envoyé à `p1` ?

```
String str = p1.sePresenter();
```

Est-ce la méthode `sePresenter` de la classe « `Personne` » ou celle redéfinie dans la sous-classe « `Salarie` » qui sera appelée ? Un simple test permet de mettre en évidence que c'est bien la méthode `sePresenter()` de la classe « `Salarie` » qui est appelée. C'est au moment de l'exécution que l'ambiguïté est levée en fonction de la classe qui a été utilisée pour créer l'instance (type dynamique).

Pour qu'une méthode ait un comportement **polymorphique**, il faut réunir les 2 conditions :

- **héritage**
- **redéfinition** de la méthode.



Pour redéfinir une méthode de la classe de base, il faut **respecter le contrat**, c'est-à-dire respecter la signature exacte de la méthode : nom de la méthode, liste des paramètres (même nombre, même ordre, même type) et le type du retour.

EXERCICES :

Exercice 15 :

A partir des classes « Personne », « Salarie » et « Client » écrites dans les précédents TP, mettre en évidence le comportement polymorphique de la méthode `sePresenter()` en codant une démonstration (un message différent selon la classe).

Exercice 16 : Le parc de véhicule (suite 1)

Nous allons faire évoluer la gestion de notre parc en prenant en compte les comportements *Demarrer ()* et *s'Arrêter()* communs à tous les véhicules.

Chacun des véhicules sait *Demarrer()* et *Arrêter()* mais le fait à sa manière : ainsi par exemple un Hélicoptère se Déplace en Volant et un bateau en Flottant.

1. Ajouter ces comportements sous forme de méthodes à votre diagramme de classe UML de l'exercice 14.
2. Adapter l'implémentation en Java de l'exercice 14.

Exercice 17 : Le parc de véhicule (suite 2)

L'entreprise qui possède le parc de véhicule fait l'acquisition d'un Hydravion et d'une voiture amphibie. Il nous faut adapter la hiérarchie de classe précédemment définie.

1. Faire évoluer votre diagramme de classe UML de l'exercice 16.
2. Adapter l'implémentation en Java de l'exercice 16.

10. Classe Abstraite et interfaces

10.1. Objectifs

- Classe abstraite
- Interface

10.2. Ce qu'il faut savoir

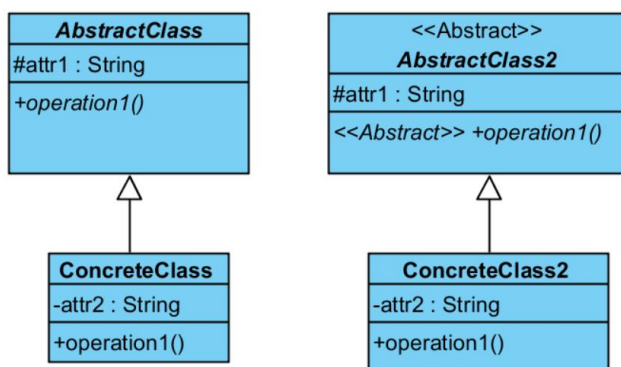
10.2.1. Classe abstraite

Dans certains cas, lors de l'élaboration d'une hiérarchie de classes, il peut être intéressant de **mettre en facteur** dans une superclasse un **comportement commun à plusieurs classes dérivées**, il peut arriver aussi que cette superclasse ne soit pas suffisamment complète pour que des objets puissent être instanciés à partir de celle-ci. C'est le cas de la classe « Véhicule » de l'exercice précédent.

La classe « Véhicule » est une **classe abstraite**. Véhicule est un concept général, les seuls véhicules qui existent sont des Voiture, Camion, Bateau, etc. Il n'existe pas de Véhicule simplement. La classe « Véhicule » comporte les propriétés et comportement que doivent avoir tous les véhicules qu'ils soient des Camion ou des Bateaux ou ... Ces particularités communes à tous les véhicules sont « récupérés » par tous les véhicules « concrets » comme les Camions et les Bateaux en héritant de la classe « Véhicule ».

Une **classe abstraite représente un concept**, les **classes concrètes** qui en héritent **réalisent ce concept**. Les classes abstraites comportent des méthodes abstraites, leur prototype (nom + arguments attendus) est spécifié mais pas leur implémentation, les sous-classes concrètes doivent redéfinir ces méthodes et en fournir une implémentation.

10.2.2. Classes abstraite en UML



En UML les éléments abstraits (*AbstractClass*, *AbstractClass2* et leur opération *operation1*) ont leur nom écrit en italique.

Comme il n'est pas toujours facile d'écrire en italique et notamment lorsque l'on dessine à la main, il est possible de créer un stéréotype « Abstract » et de l'appliquer sur les éléments abstraits.

10.2.3. Classes abstraite en Java

En Java une classe abstraite est déclarée avec le mot-clé ***abstract***, ***elle ne peut pas être instanciée, elle est destinée à être dérivée.***

```
abstract class Vehicule
{
    ...
}

class Bateau extends Vehicule
{
    ...
}

abstract class VehiculeRoulant extends Vehicule
{
    ...
}

class Voiture extends VehiculeRoulant
{
    ...
}
```

En effet, instancier un Véhicule ou un Véhicule Roulant n'a pas de sens, on ne peut instancier qu'une voiture ou un Camion (les classes concrètes dérivées).

```
Vehicule v;
v = new Vehicule(); // ERREUR DE COMPILATION
v = new Voiture();
v = new Camion();
```

10.2.4. Méthode abstraite

Quand une méthode ne peut pas être écrite dans une superclasse abstraite car son comportement est indéfini, elle est déclarée abstraite avec le mot clé ***abstract***. Une méthode abstraite n'a pas de corps.

```
abstract class Vehicule
{
    // Méthode abstraite
    public abstract void demarrer();
}

abstract class VehiculeRoulant extends Vehicule
{
    // Implémentation concrete de demarrer() (en roulant)
    public void demarrer() {
        rouler();
    }

    // Méthode abstraite (rouler comme un camion ou comme une voiture ?)
    public abstract void rouler();
}

class Camion extends VehiculeRoulant
{
    // Implémentation concrete de rouler()
    public void rouler() {
        //... rouler comme un camion ...
    }
}
```



Une méthode abstraite devra ***obligatoirement*** être redéfinie dans toutes les classes concrètes dérivées.

10.2.5. Interface

Les interfaces permettent d'uniformiser le comportement d'objets.

Une **interface** est un ensemble de **prototypes** de méthodes qui forment un **contrat**. Une interface peut être vue comme une classe abstraite "pure" dans laquelle aucune méthode n'est implémentée.

Il existe de nombreuses interfaces dans .net ou le JDK (Java) par exemple, l'interface [Comparable](#) a été décrite de la manière suivante :

Exemple d'interface :

Un *Comparable* est un objet qui est capable de se comparer à un autre pour déterminer s'il est supérieur, inférieur ou bien égal à celui-ci.

La valeur de retour est un nombre entier :

< 0 si le comparé est inférieur à l'autre

= 0 si le comparé est égal à l'autre

> 0 si le comparé est inférieur à l'autre

```
interface Comparable<T>
{
    Integer CompareTo (T autre);
}
```

Une classe qui décide d'**implémenter, de réaliser** une interface s'**engage à fournir une implémentation pour toutes les méthodes définies dans l'interface**. Cette vérification est faite à la compilation.

Dans la classe Personne on choisit de comparer les personne selon leur age :

```
class Personne implements Comparable<Personne> {
    ...
    /**
     * Implémentation de Comparable
     *
     * @param autre
     * @returns
     */
    public Integer compareTo(Personne autre) {
        if(this.age() == autre.age()) {
            return 0;
        } else if(this.age() > autre.age()) {
            return 1;
        } else {
            return -1;
        }
    }
}
```

Utilisation :

```
Terminal.afficheInt (p1.CompareTo (p2) );
```

Mais à quoi l'interface comparable peut-elle bien servir ?

Une **classe ne peut hériter que d'une seule classe**, en revanche, **une classe peut implémenter plusieurs interfaces** : c'est un biais pour implémenter l'héritage multiple.

Exercice 18 : Comparaison de salariés

- 1- Reprendre la classe Salarie écrite dans les précédents TP et implémenter l'interface Comparable qui permettra à toute instance de Salarie de se comparer à une autre instance de Salarie : Comparer 2 salariés revient à comparer leur salaire de base.
- 2- Créer une méthode trierTableau qui est capable de trier un tableau d'élément Comparable
`trierTableau(Comparable<T>[] tableauDeComparable)`

11. Collectionner des objets

11.1. Objectifs

- Utiliser des tableaux
- Utiliser des collections

11.2. Ce qu'il faut savoir

Les **tableaux** comme les **collections** **ne contiennent pas directement les objets, ils conservent des références sur les objets**, ils permettent de les manipuler comme un ensemble.

11.2.1. Les tableaux

- Un tableau stocke des éléments d'un type défini.
- Le nombre d'éléments d'un tableau doit être défini au moment de sa déclaration.
- Pour placer un objet dans un tableau, vous devez connaître son emplacement (indice compris entre 0 et la taille du tableau)

//Tableau de constantes

```
String[] tabNoms = new String[]{"AA", "AB", "AC"};
int[] tabEntiers; //tableau pas encore alloué
tabEntiers = new int[3];
```

//Tableau de personnes

```
Personne[] tabPersonnes = new Personne[10];
```

//Le tableau est vide : les références de chaque élément sont égales à null

```
tabPersonnes[0] = new Salarie("Marie", Sexe.FEMININ, new Date(82, 04, 08), 175, 22);
tabPersonnes[1] = new Salarie("Max", Sexe.MASCULIN, new Date(66, 02, 11), 165, 33);
```

//Parcours avec une boucle for

```
for (int i = 0; i < tabPersonnes.length; i++)
{
    if (tabPersonnes[i] != null)
        Terminal.ecrire(tabPersonnes[i]);
}
```

//Parcours avec une boucle avec une itération automatique

```
for (Personne pers : tabPersonnes)
{
    if (tabPersonnes[i] != null)
        Terminal.ecrire(tabPersonnes[i]);
}
```

11.2.2. Les collections

Une alternative aux tableaux est l'usage de collections. Les collections sont des classes du package `java.util.collection`

Le nombre d'éléments d'une collection n'est pas défini à l'avance, la **taille** de la collection **augmente dynamiquement** au fur et à mesure des besoins.

Les **collections** permettent :

- de stocker **un nombre quelconque d'éléments**
- d'ajouter un élément à n'importe quel endroit
- d'enlever un élément sans avoir à déplacer les autres

Il existe plusieurs implémentations des collections, on peut les classer de la manière suivante :

→ Les **listes** permettent d'utiliser un **indice**.

- `List<T>` (interface)
- `ArrayList< T>`

→ Les **dictionnaires** gèrent des **clés** pour accéder aux objets.

- `Map<K,V>` (interface)
- `HashMap<K,V>`

Une collection simple : ArrayList

→ Créer une liste (il n'est pas nécessaire d'indiquer sa capacité)

```
ArrayList liste = new ArrayList();
```

→ Ajouter des éléments à la liste

```
Personne p = new Salarie("Max", Sexe.MASCULIN, new Date(66, 02, 11"), 165, 33);
liste.Add(p);
liste.Add(new Salarie("Marie", Sexe.FEMININ, new Date(82, 04, 08"), 175, 22));
liste.Add(new Personne("Bob", Sexe.MASCULIN, new Date(68, 02, 11"), 188));
```

Une ArrayList peut contenir des objets de types différents.

→ Connaitre le nombre d'éléments de la liste

```
int nbElements = liste.size();
```

→ Savoir un objet est présent dans la liste

```
if (liste.Contains(p)) //Utilise Equals
    Terminal.ecrireStringln("La liste contient l'objet " + p);
```

→ Trouver la place d'un objet (son indice dans la liste)

```
int indice = liste.indexOf(p);
```

→ Enlever un objet dans la liste

```
liste.Remove(p);
liste.RemoveAt(indice);
```

→ Effacer tous les éléments de la liste

```
liste.clear();
```

→ Parcourir la liste avec un indice (idem tableau)

```
for (int i = 0; i < liste.Count - 1; i++)
{
    Terminal.ecrire(liste.get(i));
}
```

→ Parcourir la liste avec une boucle for avec un itérateur

```
for (Object obj in liste)
{
    Terminal.ecrire(obj);
}
```

Une collection fortement typée : List<T>



Il existe des collections génériques : **fortement typées**, qui interdisent de stocker des éléments d'un autre type que celui qui a été défini au moment de la création et dont les méthodes retournent des objets du type défini.

Créer une liste de Personnes, y ajouter des personnes

```
List<Personne> listeDePersonnes = new ArrayList<Personne>();
listeDePersonnes.Add(new Personne("GREGOIRE", "Ghislaine", 25));
listeDePersonnes.Add(new Etudiant("gregoire", "Laurie", 20, "Commerce"));
listeDePersonnes.Add(new Object()); ;//interdit par les listes fortement typées
```

Quand on parcourt la liste de Personne, on récupère un objet de type Personne et non de type Object, on a ainsi accès aux méthodes de la classe Personne.

```
for (Personne pers : listeDePersonnes)
{
    Terminal.ecrire(pers);
}

for (int i = 0; i < listeDePersonnes.Count - 1; i++)
{
    Terminal.ecrire(liste.get(i).getNom());
}
```

Une collection de type dictionnaire : Map <K, V>

Les collections de type Dictionnaire permettent d'associer une **clé** à chaque objet pour le retrouver plus facilement.

→ Créer une Map de départements

```
Map<Integer, String> mapDepartements = new HashMap<Integer, string>();
```

→ Ajouter des départements (clé, valeur)

```
mapDepartements.put(12, "AVEYRON");  
mapDepartements.put(81, "TARN");  
mapDepartements.put(34, "HERAULT");
```

→ Rechercher un élément à partir de sa clé

```
if (mapDepartements.containsKey(12))  
    Terminal.ecrire(mapDepartements.get(12));
```

→ Parcourir les valeurs du dictionnaire

```
for (Entry<Integer, String> deptEntry : mapDepartements.entrySet())  
{  
    Terminal.ecrireStringln(deptEntry.getKey() + " " + deptEntry.getValue());  
}
```

Utiliser un itérateur pour parcourir les éléments

Les collections ont un comportement commun : elles implémentent l'interface `Iterable`. Elles possèdent toutes une méthode `iterator()` qui renvoie un itérateur (type `Iterator`).

Cet objet permet de parcourir les éléments de tous les types de collections de la même manière, sans avoir à connaître comment la collection stocke ces éléments :

- Lors de sa création, l'énumérateur se place **avant le premier élément**
- La méthode `boolean hasNext()` retourne `true` s'il existe encore des éléments à parcourir et `false` sinon.
- La méthode `next()` déplace le curseur sur l'élément suivant et le retourne.

La procédure ci-dessous affiche le contenu de toute structure de données passée en paramètre tout autant qu'elle implémente l'interface `Iterable`.

```
public static void printValues (Iterable iterable)  
{  
    Iterator it = iterable.iterator();  
    while (it.hasNext())  
    {  
        Terminal.ecrire(it.next());  
    }  
}
```



Ici les éléments retournés sont de type `Object`, on devra caster ces objets pour accéder à leurs méthodes propres (si on est sûr de leur type).

12. Exceptions

12.1. Objectifs

- Lever une exception (throw)
- Intercepter et traiter une exception (try/catch)
- Créer ses propres classes d'exception

12.2. Ce qu'il faut savoir

Une exception est comme une alerte créée et lancée par une méthode d'une classe quand une erreur survient au moment de l'exécution.

Dès qu'une exception est lancée, elle se propage dans la pile d'appel de la manière suivante :

- L'exécution normale de la méthode est interrompue
- Un gestionnaire d'exceptions (bloc try/catch) est recherché dans le bloc d'instruction courant. S'il n'est pas trouvé, la recherche se fait dans la fonction appelante et ainsi de suite jusqu'à remonter au bloc Main. Si aucun bloc try/catch n'est trouvé, le programme se termine anormalement.

12.2.1. La classe Exception et ses filles

Les exceptions sont des objets issus de la classe `java.lang.Exception` ou de l'une de ses sous-classes.

La classe `Exception` contient une propriété remarquable : le **message**, c'est lui qui est passé en paramètre au constructeur pour décrire l'erreur lors de la création d'une instance et qui sera affiché par la méthode **`ToString()`**.

12.2.2. Lever une exception

En Java, si une méthode peut lever une exception, elle doit le déclarer dans son prototype :

Si on considère la classe `Personne`, la valeur attribuée à la propriété `taille` devrait être contrôlée et se situer dans une fourchette de 100 à 220. Il faut donc revoir le constructeur et la méthode `set` de la propriété `taille` pour lever une exception quand la valeur donnée en paramètre est hors limite.

Dans la classe `Personne` :

```
public Personne(String nom, Sexe sexe, Date dateNaissance, Integer taille, String profession) {
    super();
    this.nom = nom;
    this.sexe = sexe;
    this.dateNaissance = dateNaissance;
    setTaille(taille);
    this.profession = profession;
}

//...

public void setTaille(Integer taille) throws IllegalArgumentException{
    if(taille < 100 || taille > 220) {
        throw new IllegalArgumentException("Taille invalide (valeur doit être entre 100 et 220)");
    }
    this.taille = taille;
}
```

L'exception `IllegalArgumentException` est déclarée dans le prototype comme pouvant être levée.

L'instruction `throw` permet de lever l'exception.

Une exception est une instance de la classe `Exception` ou de l'une de ses sous-classes.



L'instruction **`throw`** effectue **un branchement inconditionnel vers le bloc `catch`** correspondant au traitement de cette exception. En conséquence, les instructions qui suivent `throw` ne seront pas exécutées.

12.2.3. Intercepter et traiter une exception

Lorsqu'une exception n'est pas interceptée elle se propage, elle remonte la pile d'appels et le programme se termine anormalement.

Afin d'intercepter (gérer) une exception, il faut encadrer l'instruction susceptible de lever une exception par un bloc `try` suivi d'un ou plusieurs blocs `catch` qui permettront d'intercepter et de traiter les différents types d'exception susceptibles de se produire.

En Java si une méthode pouvant lever une exception est appelé l'appel doit être dans un bloc `try/catch` ou bien la méthode appelante doit déclarer dans son prototype la possibilité de lever cette exception.

```
try
{
    // Une ou plusieurs instructions peuvent lever une exception
}
catch (TypeException variable)
{
    // Actions à effectuer dans le cas d'une exception du type TypeException
}
finally
{
    // Le bloc finally est optionnel
    // Instructions exécutées dans tous les cas
}
```

Un bloc `try` peut être suivi de plusieurs blocs `catch` attrapant différents types d'exception et éventuellement d'un bloc `finally` toujours exécuté qu'il y ait eu une exception ou pas.



Les blocs `catch` agissent comme des filtres. L'ordre des blocs **`catch`** est important car ils sont évalués dans leur ordre d'apparition. Il est incorrect de placer un bloc `catch` d'une classe de base avant le bloc `catch` d'une sous-classe.

Revenons à notre exemple, dans le bloc `Main` :

```
Personne p = new Personne("ESSAI", "Test", -1);
```

Cette instruction lève une exception. Il faut donc placer cette instruction dans un bloc `try/catch` pour la capturer et la traiter.

```
try
{
    Personne p = new Personne("ESSAI", "Test", -1);
}
```

```
catch (IllegalArgumentException e)
{
    e.printStackTrace();
}
```

Quand une exception survient et qu'elle est traitée, elle est capturée et le programme continue normalement.

12.2.4. Créer une nouvelle classe Exception

Lancer une exception signale une erreur particulière. Si aucune classe Exception prédéfinie ne convient ou n'est suffisamment précise, ou si l'exception doit comporter des informations supplémentaires, il est possible de créer une nouvelle classe d'exception.

Pour cela, il faut dériver la classe `Exception` ou l'une de ses sous-classes. Par convention, toutes ces sous-classes ont un nom se terminant par `Exception`.

La création d'une nouvelle classe d'exception s'impose quand on veut filtrer, identifier les exceptions propres à une classe donnée.

3 étapes :

- Dériver la classe `Exception` pour créer une nouvelle classe par héritage.
- Coder un constructeur passer en argument les informations que l'on souhaite afficher dans le message d'erreur (le constructeur doit appeler un des constructeurs de la classe de base).

Cette nouvelle classe peut être maintenant être utilisée comme n'importe quelle autre classe d'exception.

Index

A

abstract, 58, 59
ArrayList Voir collections

C

casting, 53
Classe, 15, 16
Classe abstraite, 57
Classe dérivée Voir classe Fille, sous-classe
Classe Fille, 46
Collections, 63
Collections génériques, 64

E

Encapsulation, 30
Exception, 66

G

Garbage Collector, 44
Get, 28

H

Héritage, 46, 55

I

Implémenter Voir Interface

Interface, 60

M

Méthode abstraite, 59

P

Polymorphisme, 55
Private, 28
Propriété de classe, 42
Protected, 28, 50
Public, 28

R

Redéfinition, 55

S

sealed Voir Classe Finale
Set, 29
static Voir Propriété de Classe
Surcharge, 20

T

Tableau, 62
this, 29
throw, 67